

Module 1 — Replay

A Hands-On Tutorial on the Underwater Acoustic Channel Library

Zhengnan Li · Milica Stojanovic

Monday, September 21, 2026

OCEANS 2026 · Monterey Conference Center, Monterey, CA

09:30 – 10:10 · Module 1 of 5 · bring your laptop

Where we are

08:30 – 09:30	Introduction
09:30 – 10:10	Module 1 — Replay
10:10 – 10:30	Coffee
10:30 – 10:50	Module 2 — Ocean noise
10:50 – 11:10	Module 3 — Unpack and visualize
11:10 – 11:40	Module 4 — The Library at a Glance
11:40 – 12:20	Module 5 — The estimate repository
12:20 – 12:30	Wrap-up and Q&A

Half of this module is yours

Two blocks of ten minutes where you type and run, not watch. Slides 19 and 27. Laptops open now; if the toolbox is not installed, fix that during the first block and not during the walkthrough.

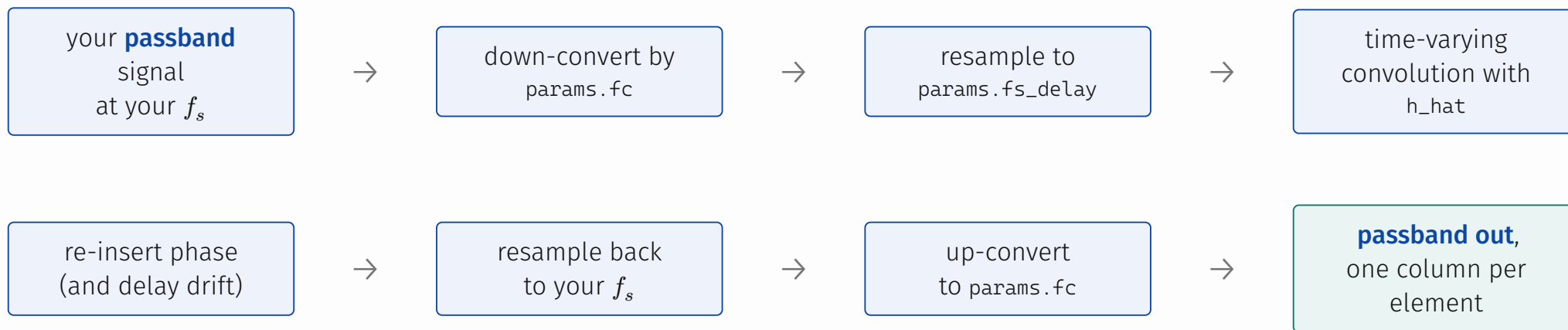
By 10:10 you will have

1. Loaded a measured channel file and looked inside it, without guessing what is in it.
2. Built a BPSK probe that **fits** the channel — the right carrier, the right bandwidth, the right length.
3. Driven it through **blue** and got back a received waveform on three hydrophones.
4. Recognized the ocean in three plots: the waveform, the matched-filter output, the spectrum.
5. Done it again with a channel and a probe of your own choosing, and explained why the answer changed.

What you will **not** do: equalize, synchronize, decode. `replay` gives you the received signal. What you do with it is your receiver's job, and your research.

What replay does

One function, seven steps, all of them internal



Read that first and last box again

You hand it **passband**, you get **passband** back. Everything in between — the baseband conversion, the resampling, the phase and delay bookkeeping — happens inside, using the numbers stored in the file. You never write a down-converter.

The call, argument by argument

MATLAB / Octave

```
y = replay(x, fs, array_index, channel)
y = replay(x, fs, array_index, channel, start)
```

Python

```
y = replay(x, fs, array_index, channel)
y = replay(x, fs, array_index, channel, start=n)
```

Argument	What it is	Watch out
x	Real passband signal, one column	Not baseband, not complex.
fs	Your sampling rate [Hz]	Yours, not the channel's. replay resamples.
array_index	Which hydrophones to receive on	1-based in MATLAB, 0-based in Python.
channel	The loaded file	load(...) / load_channel(...).
start	Where in the recording to begin	Optional — omit it and the start is random .

Returns a [time, M] real passband array, a little **longer** than your input.

Three rules before you type anything

1. **Your signal fits inside the channel's band.** `blue` is 5 kHz wide at 13 kHz. A 12 kHz-wide signal will not error — it will quietly return something that is not a measurement of anything.
2. **Your signal is shorter than the recording.** `blue` is 52 s long. Longer input and `replay` stops you with `Duration of the input signal ... should be no larger than ...`
3. **The carrier comes from the file, not from your memory.** Read `params.fc`. Every channel has its own, and the slide deck is a summary while the file is the record.

A fourth rule, for later

Pass `start` when the number matters. Without it, every run draws a different stretch of ocean and your results move around for no reason you can see.

The walkthrough



Step 0 — two files, in the folder you are working in

From the Zenodo record, doi:10.5281/zenodo.21287414:

File	Size	What it is
blue_1.mat	206 MB	One recording from MACE'10 — 52 s, 12 elements
blue_noise.mat	0.2 MB	The matching ambient-noise model for the site

If you do not have them

I have a USB drive that has the complete dataset. Do not start a 206 MB download now — a hundred of them at once is why we asked you to do it at home.

MATLAB's `load` reads the whole 206 MB into memory. `load_channel` does not — it reads lazily, so Python will feel faster to start and no slower to run.

Step 1 — load it, and look inside

MATLAB / Octave

```
channel = load('blue_1.mat');  
noise   = load('blue_noise.mat');
```

```
disp(channel)  
disp(channel.params)  
size(channel.h_hat)
```

Python

```
from uwa_channels import load_channel,  
load_noise  
channel = load_channel("blue_1.mat")  
noise   = load_noise("blue_noise.mat")
```

```
print(list(channel.keys()))  
print(list(channel["params"].keys()))  
print(channel["h_hat"].shape)
```

Do this every time you meet a new channel

Two things decide how you use the file, and both are visible in thirty seconds: the **shape** of `h_hat`, and whether the phase field is `phi_hat` or `theta_hat`.

What is actually in `blue_1.mat`

```
h_hat      320 x 12 x 2553    complex
phi_hat     12 x 510600      radians
params      fs_delay  9765.625 Hz
             fs_time   48.828 Hz
             fc        13000   Hz
meta        18 fields (codename, optimizer,
             element_spacing, ...)
version     1.0
```

Everything you need is in those six lines:

- **320 delay taps** at 9765.625 Hz = **32.8 ms** of delay coverage.
- **12 elements.** `array_index` may not exceed that.
- **2553 snapshots** at 48.828 Hz = **52.3 s** of recording. Your signal must be shorter.
- **`fc = 13 kHz`.** Build your carrier from this number.
- **`phi_hat` is present**, so the taps are drift-free and the motion lives in the phase. `replay` puts it back; you do nothing.

The one thing that surprises Python users

MATLAB

```
size(channel.h_hat)
```

```
320    12    2553
```

[delay, receiver, time] — the order the documentation states, and complex numbers are complex numbers.

Python

```
channel["h_hat"].shape
```

```
(2553, 12, 320)
```

HDF5 stores MATLAB arrays **reversed**, so you get (time, receiver, delay). And the dtype is a compound ('real', 'imag'), not complex128.

This bites once, then never again

If you are writing your own reader, `np.array(f["h_hat"]["real"]) + 1j*np.array(f["h_hat"]["imag"])` and remember the axis order. If you are calling `replay`, it already knows — pass the `load_channel` result straight in.

Step 2 — four numbers

MATLAB / Octave

```
fs = 48e3;           % yours
fc = channel.params.fc; % 13 kHz, from the
file
R = 4e3;             % inside B = 5 kHz
array_index = [1, 6, 12]; % 1-based
```

Python

```
fs = 48e3           # yours
fc = channel["params"]["fc"][0, 0] # 13 kHz
R = 4e3             # inside B
array_index = [0, 5, 11]          # 0-based
```

- f_s is **your** sampling rate. 48 kHz is convenient — it is a sound-card rate and a clean multiple of R . The channel's own rate is 9765.625 Hz and `replay` handles the conversion.
- $R = 4$ ksym/s with root-raised-cosine-free rectangular shaping occupies about 4 kHz, comfortably inside **blue**'s 5 kHz. Push it to 5 and you are on the edge; to 12 and you are outside.
- Those two index lists name the **same three hydrophones**: top, middle, bottom.

Step 3 — build the probe

MATLAB / Octave

```
rng(42);
sym = 2 * randi([0, 1], 1023, 1) - 1;
bb = resample(repmat(sym, 8, 1), fs/R, 1);
pb = real(bb .* exp(2j*pi*fc* ...
    (0:length(bb)-1).'/fs));
x = [zeros(fs/10, 1); pb; zeros(fs/10, 1)];
```

Python

```
rng = np.random.default_rng(42)
sym = rng.choice([-1.0, 1.0], size=1023)
bb = sg.resample_poly(np.tile(sym, 8),
    int(fs), int(R))
pb = np.real(bb * np.exp(
    2j*np.pi*fc*np.arange(len(bb))/fs))
x = np.concatenate([np.zeros(int(fs/10)),
    pb, np.zeros(int(fs/10))])
```

- **1023 BPSK symbols**, a PN period, repeated **8 times** → 2.05 s of probe. Well under 52 s.
- **Upsample** by $f_s/R = 12$, then put it on the carrier and take the real part. That is your passband signal.
- **A guard of 0.1 s** at each end. You want somewhere for the channel's tail to land, and it makes the burst easy to find in the plot.
- **Seed the generator.** You need the same symbols back at the receiver to correlate against.

Step 4 — replay

MATLAB / Octave

```
y = replay(x, fs, array_index, channel,  
20000);  
size(y)    % 109383 x 3
```

Python

```
y = replay(x, fs, array_index, channel,  
start=20000)  
y.shape    # (109383, 3)
```

That is the whole thing. One call.

- **Three columns**, because you asked for three elements.
- **109 383 rows** against your 107 808 — the output is longer by the channel's delay spread plus a little interpolation buffer.
- **start = 20000** picks the sample of the recording to begin at, in the delay-domain clock (9765.625 Hz), so 20 000 is about 2.0 s into the 52 s record. Omit it and you get a different stretch of ocean every run.

Step 5 — noise, and SNR

MATLAB / Octave

```
w = noisegen(size(y), fs, array_index, noise);  
  
r = y + 0.05 * w;
```

Python

```
w = noisegen(y.shape, fs, array_index, noise)  
  
r = y + 0.05 * w
```

Why $r = y + 0.05 * w$?

The power of y about M , where M is the number of the channels. The noise w also has a power of about M .

Module 2 is about what is inside noisegen. Today it is one line.

The whole thing — MATLAB

```
channel = load('blue_1.mat');
noise   = load('blue_noise.mat');

fs = 48e3;  fc = channel.params.fc;  R = 4e3;  array_index = [1, 6, 12];

rng(42);
sym = 2 * randi([0, 1], 1023, 1) - 1;
bb  = resample(repmat(sym, 8, 1), fs/R, 1);
pb  = real(bb .* exp(2j*pi*fc*(0:length(bb)-1).'/fs));
x   = [zeros(fs/10, 1); pb; zeros(fs/10, 1)];

y = replay(x, fs, array_index, channel, 20000);
w = noisegen(size(y), fs, array_index, noise);
r = y + 0.05 * w;

t = (0:size(r,1)-1)/fs;
figure, plot(t, r), xlabel('Time [s]'), ylabel('Received signal')

v   = r(:,1) .* exp(-2j*pi*fc*(0:size(r,1)-1).'/fs);
ref = resample(sym(1:128), fs/R, 1);
figure, plot(abs(xcorr(v, ref))), xlabel('Samples'), ylabel('Xcorr')

figure, pwelch(r, kaiser(1024,5), 512, 4096, fs), xlim([fc-R, fc+R]/1e3)
```

The whole thing — Python

```
import numpy as np, scipy.signal as sg, matplotlib.pyplot as plt
from uwa_channels import replay, noisegen, load_channel, load_noise

channel = load_channel("blue_1.mat")
noise   = load_noise("blue_noise.mat")

fs = 48e3; fc = channel["params"]["fc"][0, 0]; R = 4e3; array_index = [0, 5, 11]

rng = np.random.default_rng(42)
sym = rng.choice([-1.0, 1.0], size=1023)
bb = sg.resample_poly(np.tile(sym, 8), int(fs), int(R))
pb = np.real(bb * np.exp(2j*np.pi*fc*np.arange(len(bb))/fs))
x = np.concatenate([np.zeros(int(fs/10)), pb, np.zeros(int(fs/10))])

y = replay(x, fs, array_index, channel, start=20000)
w = noisegen(y.shape, fs, array_index, noise)
r = y + 0.05 * w

t = np.arange(len(r))/fs
plt.figure(); plt.plot(t, r); plt.xlabel("Time [s]"); plt.ylabel("Received signal")

v = r[:,0] * np.exp(-2j*np.pi*fc*np.arange(len(r))/fs)
ref = sg.resample_poly(sym[:128], int(fs), int(R))
plt.figure(); plt.plot(np.abs(np.correlate(v, ref, "full"))); plt.ylabel("Xcorr")

plt.figure(); plt.psd(r[:,0], NFFT=4096, Fs=fs); plt.xlim(fc-R, fc+R); plt.show()
```

Your turn



Hands-on 1 — make it run

Type it, run it, get three figures

HANDS-ON · 10 minutes

1. Open MATLAB or Python in the folder holding `blue_1.mat` and `blue_noise.mat`.
2. Type the script from the previous two slides. **Type it, do not paste it.**
3. Before you plot, check that `size(y)` is 109383×3 and `sigma` is a small positive number.
4. Make the three figures, and keep them open.
5. **Finished early?** Replay through one element, then all twelve. Watch the clock.

We are back at 09:46

Raise a hand if you are stuck. The next slide lists what goes wrong; it stays up.

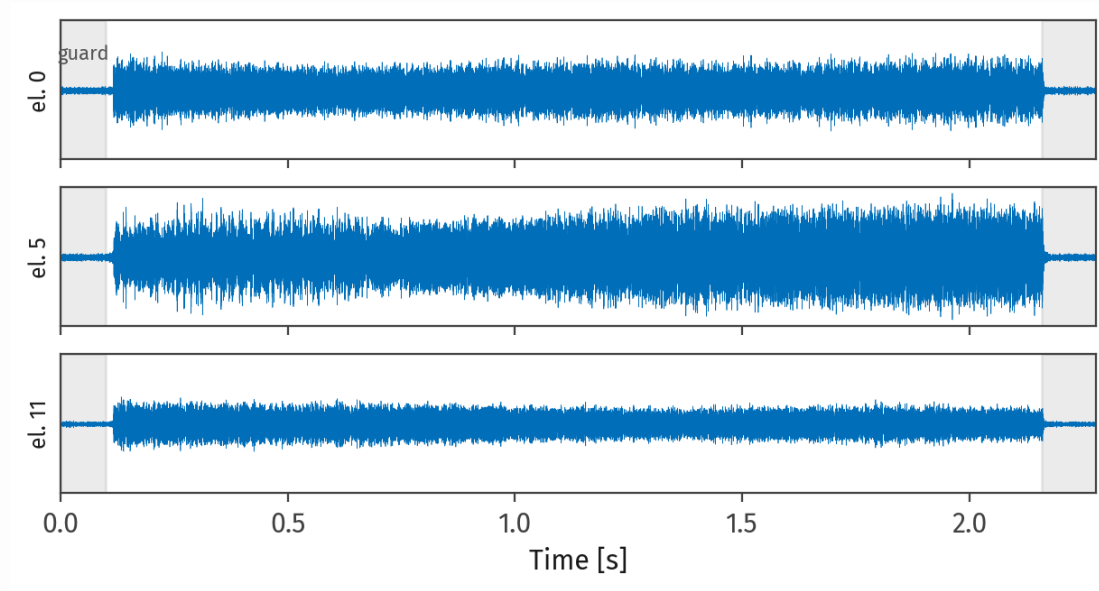
If it does not run

What you see	What it means
Undefined function 'replay'	Toolbox not on the path. Run <code>install</code> from the clone, or install the <code>.mltbx</code> .
No module named 'uwa_channels'	<code>pip install uwa-channels</code> in the environment you are actually running.
Undefined function 'resample'	MATLAB: Signal Processing Toolbox. Octave: <code>pkg load signal statistics</code> .
Duration of the input signal...	Your probe is longer than the recording. Lower the repeat count.
<code>array_index</code> must be...	Out of range or duplicated. blue has 12 elements; MATLAB counts from 1, Python from 0.
Unable to open file / <code>OSError</code>	The <code>.mat</code> is not in the working directory. In Python, open it with <code>load_channel</code> , not <code>h5py.File</code> .
It runs, but every plot is noise	Check <code>fc</code> . If you typed a carrier the channel does not have, you are outside the band.
It is very slow	You asked for all 12 elements. Start with three.

Reading the output



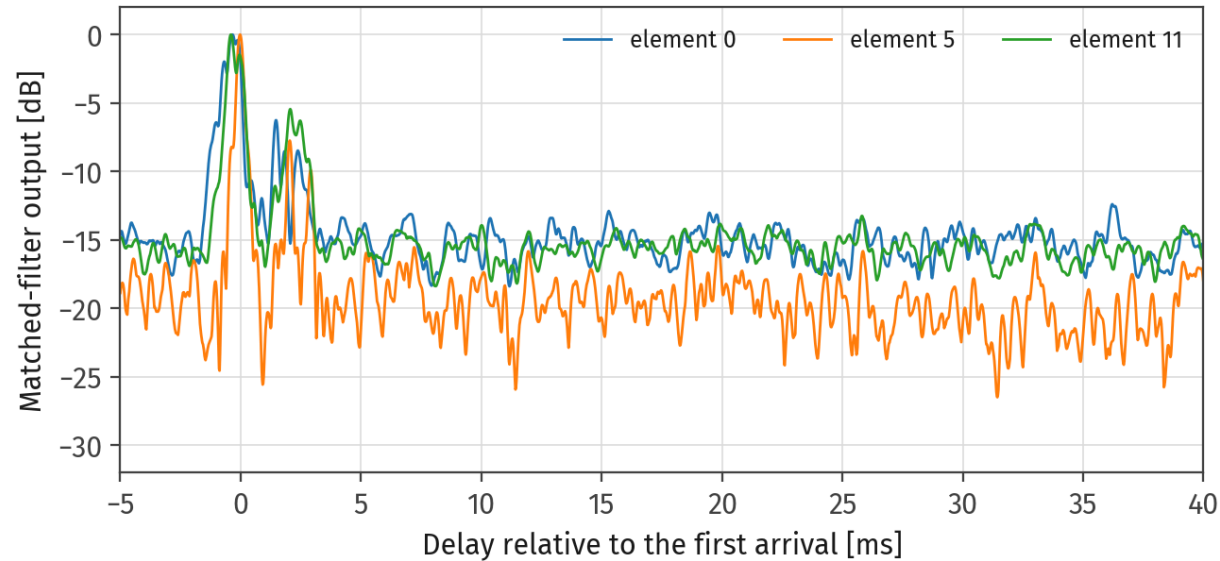
The waveform — what came back



Elements 0, 5 and 11 of **blue**, 20 dB SNR. Same transmission, same instant.

The guard bands are still there, and the burst sits between them. What matters is that the three rows are **different**: different amplitude, different fading. That difference is the array, and it is why spatial processing works.

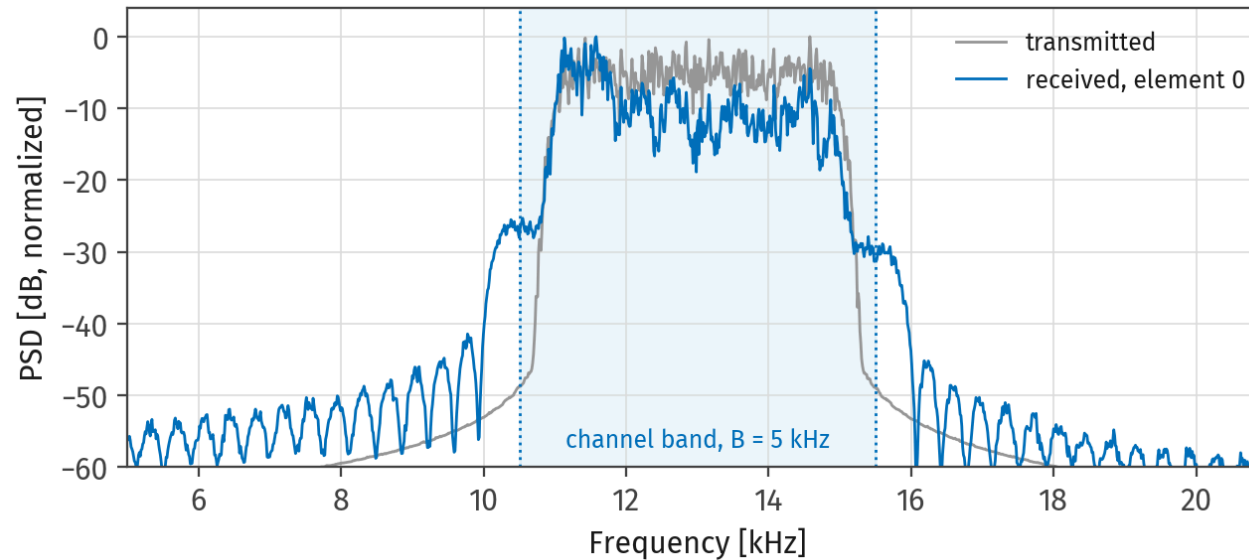
The matched filter — this is the channel



Matched-filter output against the transmitted PN sequence, normalized to the first arrival.

A first arrival at 0 ms, a second cluster at 1–3 ms about 5 dB down, and then a floor. That is **blue's** multipath, measured through a signal **you** built. Module 3 opens the same channel up directly, with `unpack`, and you get the same structure without a probe.

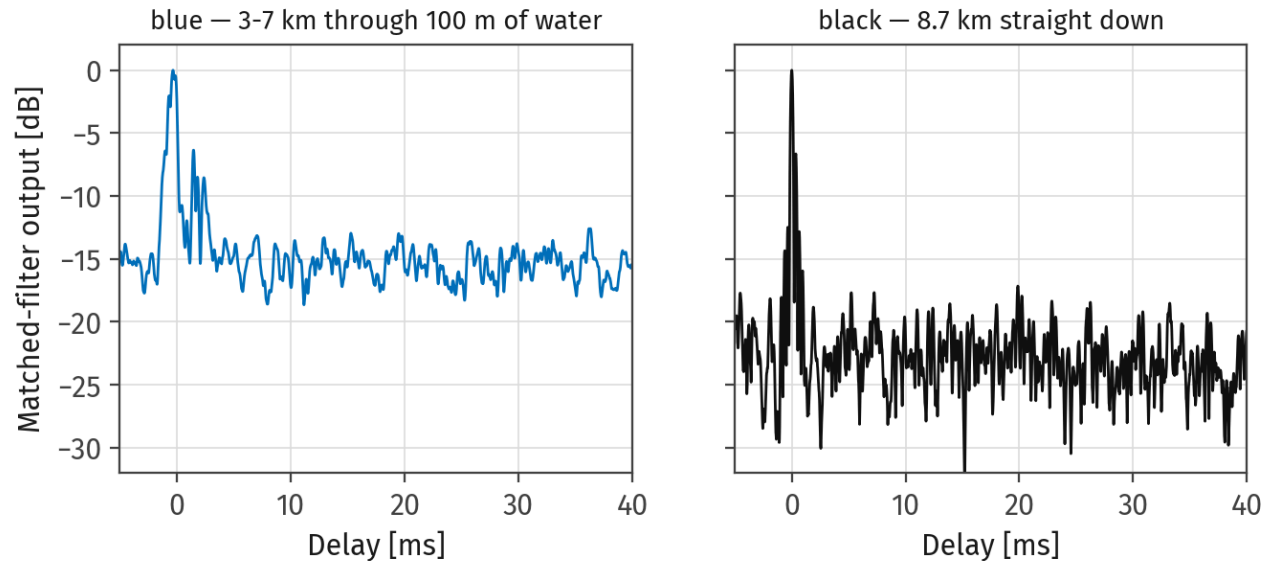
The spectrum — did you stay inside the band?



Transmitted (gray) and received (blue) power spectral density, element 0.

Two things to read. **Inside** the band, the channel has carved deep notches into a flat transmit spectrum — that is frequency-selective fading, and it is why you need an equalizer. **Outside** the band, everything you see is the noise model, which is band-limited to the experiment's band because that is all that was measured.

Same probe, different ocean



The identical 1023-symbol BPSK probe, replayed through two channels.

blue spreads the arrival over about 3 ms. **black** — 8.7 km straight down through the Mariana Trench — returns a single spike. Nothing changed but the file. That is the whole argument for a library of measured channels rather than one model.

Your turn, again



Hands-on 2 — your probe, your channel

Change one thing at a time, and explain what happened

HANDS-ON · 10 minutes

- 1. Move along the array.** Replace `array_index` with three different elements — adjacent ones, then the two extremes. Re-plot the matched filter. Which elements look alike, and why?
- 2. Change the probe.** Pick one: double the symbol rate to 8 ksym/s, or halve it to 2, or go to QPSK. Keep it inside 5 kHz. What happened to the resolution of the arrivals?
- 3. Change the channel.** Load a different file and re-run. Remember to take f_c , and the number of elements, from the new file.
- 4. Stretch.** Run the same call twice with `start` fixed, then twice with `start` omitted. Explain the difference to your neighbor.

We are back at 10:02

One question to have an answer to: **what did you change, and did the plot move the way you expected?**

Changing the symbol rate

- R must divide f_s evenly, or use `resample(x, p, q)` with a proper ratio.
- Re-make `ref` at the new rate too, or the correlation will not peak.

Going to QPSK

```
sym = (randi([0,1],1023,1)*2-1 + ...  
       1j*(randi([0,1],1023,1)*2-1))/sqrt(2);
```

Everything downstream is unchanged — `bb` is already complex.

Changing the channel

- **black**: $f_c = 18$ kHz, $B = 12.5$ kHz, 8 elements, 36 s. Raise f_s to 96 kHz.
- **green**: $f_c = 6$ kHz, $B = 4.5$ kHz, and remember its 64 “elements” are **repeats in time**, not an array.
- **red**: $f_c = 25$ kHz, only 3 elements, and impulsive noise.

If nothing correlates Print f_c , f_s , R and `size(y)`. One of the four is wrong; it is usually f_c .

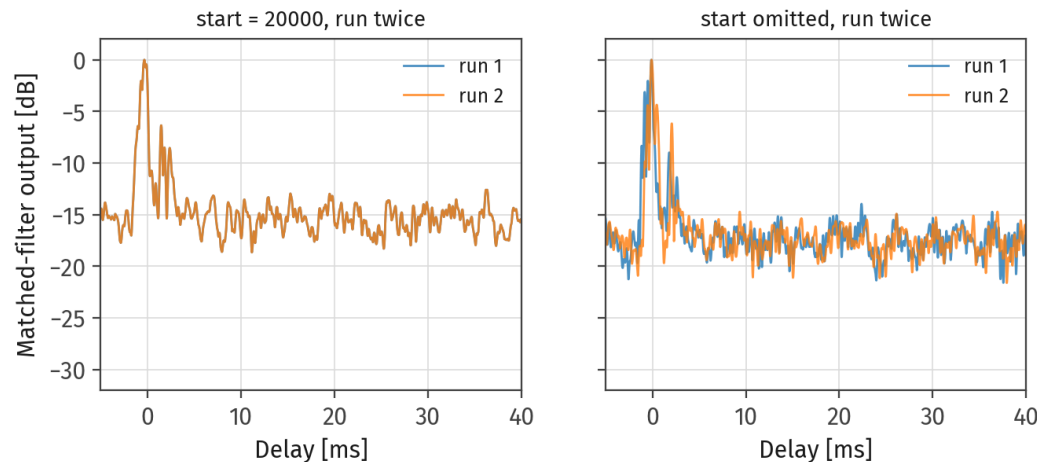
What just happened



What you should have seen

- **Adjacent elements look alike; the extremes do not.** **blue**'s array spans 1.3 m at 12 cm spacing, so neighboring hydrophones see nearly the same arrivals and the ends see different ones. That is exactly the correlation a beamformer exploits.
- **A higher symbol rate resolves more arrivals, up to the band edge.** Delay resolution is $\frac{1}{B}$, so 5 kHz of bandwidth resolves 0.2 ms. Doubling R from 4 to 8 ksym/s buys you nothing once you leave the band — and costs you everything.
- **A different channel is a different world.** Not a different parameter setting.
- **start decides whether you ran an experiment or a demo.**

start — the difference between a demo and an experiment



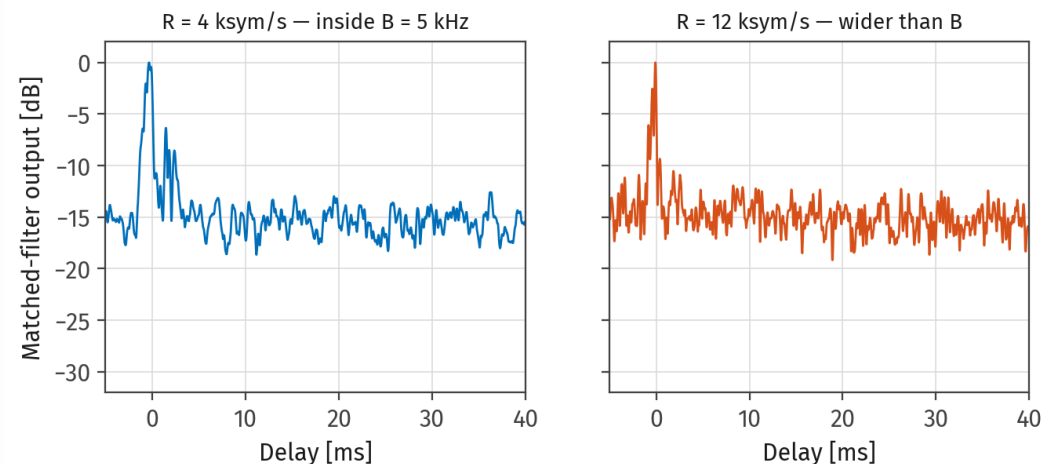
The same call, twice, with and without `start`.

With `start` fixed the two runs are **bit-identical**. Without it you replay a different stretch of ocean each time, and your BER curve wobbles for reasons that have nothing to do with your receiver.

The rule

Sweeping over channel realizations is a legitimate experiment — but **choose** the starts, write them down, report them.

Out of band does not raise an error



blue, $B = 5$ kHz. Left: a 4 ksym/s probe. Right: a 12 ksym/s probe.

The right-hand plot looks fine: a sharp peak, a plausible floor, no warning printed. And it is meaningless — seven of its twelve kilohertz were never measured at sea.

This is the failure mode to remember

Every other mistake in this module announces itself. This one does not.

MATLAB and Python: four differences that matter

	MATLAB / Octave	Python
Element numbering	<code>array_index = [1, 6, 12]</code>	<code>array_index = [0, 5, 11]</code>
Opening the file	<code>load</code> — reads all 206 MB	<code>load_channel</code> — lazy
Shape of <code>h_hat</code>	<code>[320, 12, 2553]</code> <code>[delay, receiver, time]</code>	<code>(2553, 12, 320)</code> HDF5 reverses the axes
Complex storage	Native complex	Compound dtype with <code>real</code> / <code>imag</code> fields
The optional start	<code>replay(..., channel, 20000)</code>	<code>replay(..., channel, start=20000)</code>

Everything else is the same on purpose: same function names, same argument order, same returned shape. A script ported between them should differ only in those rows.

What replay does not do

- **It does not synchronize.** You get the whole record including your guard bands. Finding the burst is your receiver's first job.
- **It does not equalize, and it does not decode.** No channel estimate is handed back.
- **It does not add noise.** That is noisegen, and it is deliberately a separate call so you can set the SNR yourself.
- **It does not widen the band or lengthen the record.** Those are properties of the sea trial, not of the software.
- **It does not show you the channel.** For the impulse response itself, unpack — Module 3.

What it does do

Take any waveform you can imagine, and tell you what the ocean would have done to it — reproducibly, from a real recording, in one line.

Recap

1. `replay(x, fs, array_index, channel[, start])` — passband in, passband out, one column per element.
2. Take `fc` and the element count **from the file**. Keep your bandwidth inside the channel's and your signal shorter than the record.
3. Set the noise level from a target SNR. `replay` does not normalize.
4. Pass `start` when the number matters.
5. Read the answer in three plots: the waveform shows the array, the matched filter shows the multipath, the spectrum shows whether you stayed legal.

The full scripts are in `examples/example_replay.m` and `examples/example_replay.py`, and the reference solutions for both hands-on blocks are on the tutorial site.

Module 2 — Ocean noise

Now open up that one line you wrote, `noisegen`.

Module 2 — Ocean Noise

A Hands-On Tutorial on the Underwater Acoustic Channel Library

Zhengnan Li · Milica Stojanovic

Monday, September 21, 2026

OCEANS 2026 · Monterey Conference Center, Monterey, CA

10:30 – 10:50 · Module 2 of 5

Where we are

09:30 – 10:10	Module 1 — Replay
10:10 – 10:30	Coffee
10:30 – 10:50	Module 2 — Ocean noise
10:50 – 11:10	Module 3 — Unpack and visualize
11:10 – 11:40	Module 4 — The Library at a Glance
11:40 – 12:20	Module 5 — The <code>estimate</code> repository
12:20 – 12:30	Wrap-up and Q&A

Three configurations, three diagnostics

Twenty minutes, one function, three ways to call it. After each one there is a two-minute check you can run on your own laptop — keep the terminal from Module 1 open.

The line you already wrote

In Module 1 this was one line and we moved on:

MATLAB / Octave

```
w = noisegen(size(y), fs, array_index, noise);
```

Python

```
w = noisegen(y.shape, fs, array_index, noise)
```

Why it is worth twenty minutes

Every receiver you build assumes something about the noise. Almost every textbook assumes white and Gaussian. **The ocean is neither.** It is colored, it is correlated across your array, and in warm shallow water it is not even Gaussian.

Each of those three assumptions, broken, costs you performance you will otherwise blame on your equalizer.

Three ways to call it



One function, three configurations

Configuration	Call	What you get
Generic	<code>noisegen(size, fs)</code>	Colored Gaussian, -17 dB/decade, independent across elements
Site-specific Gaussian	<code>noisegen(size, fs, idx, noise)</code> with $\alpha = 2$	The measured spectrum and the measured spatial correlation
Impulsive	<code>noisegen(size, fs, idx, noise)</code> with $\alpha < 2$	The same, plus heavy tails — snapping shrimp

The last two are the **same call**. `noisegen` reads α out of the file and picks the generator for you; you never choose between them by hand.

blue yellow purple green black pink — Gaussian **red** — impulsive **brown** — no site model, use the generic one

What is in a noise file

Field	Meaning
Fs	Rate the statistics were measured at [Hz]
R	Bandwidth of the recorded noise [Hz] — not a symbol rate, despite the name
alpha	Characteristic exponent. 2 Gaussian, < 2 impulsive
beta	Mixing coefficients, [M, M, L+1]
fc	Center frequency [Hz]
version	Format version

Read from the released files:

```
blue_noise  alpha 2.0  fc 13000  Fs 48000
              R 4882.8  beta (12,12,129)
              -> M = 12, L = 128
red_noise   alpha 1.7  fc 25000  Fs 96000
              R 9600    beta (3,3,65)
              -> M = 3,  L = 64
```

L is the longest lag with appreciable covariance. It is 64 for **red** and **purple**, 128 for **blue**, **black**, **yellow** and **pink**, and **512** for **green** — the fjord's reverberation shows up in the noise too.

The one equation

$$\hat{n}_i(nT_s) = \sum_{j=0}^{M-1} \sum_{k=0}^L \beta_{ij}(kT_s) \eta_j(nT_s - kT_s)$$

Draw M independent white sequences η_j , then filter them **across elements and across lags** with one set of coefficients. That single operation buys all three properties:

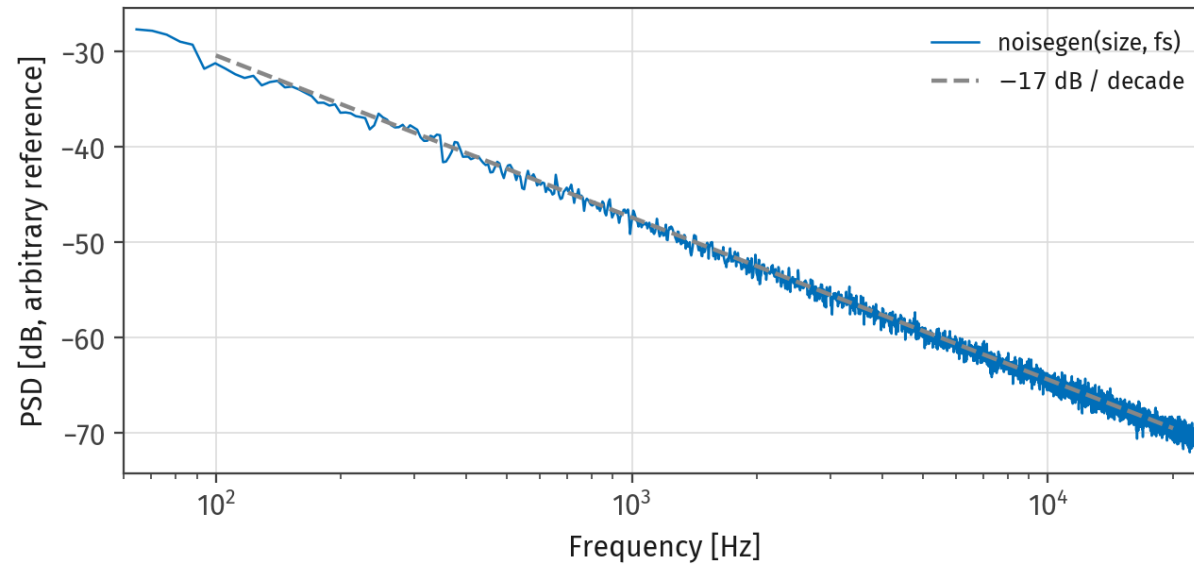
- **Color** comes from the lag sum over k — the β act as a filter in time.
- **Spatial correlation** comes from the element sum over j — each output mixes every input.
- **Impulsiveness** comes from the draw: $\eta_j \sim \mathcal{N}(0, 1)$ when $\alpha = 2$, and $\eta_j \sim \mathcal{S}_\alpha(0, 1/\sqrt{2})$ when $\alpha < 2$.

The same machinery, one substitution

A symmetric α -stable draw with scale $1/\sqrt{2}$ **is** a standard Gaussian when $\alpha = 2$. That is why one function covers both, and why the β are estimated the same way for **red** as for everything else.

Configuration 1 — the generic model

noisegen(size, fs) — when you have no site model



One element of `noisegen((218, 4), 48e3)`, Welch estimate, against the nominal slope.

No file, no array index. Power decaying log-linearly with frequency — nominally 17 dB per decade — and independent across elements. It is the standard ambient-noise shape, and it is what **brown** has to use.

Diagnostic 1 — measure the slope yourself

Two minutes

Generate it, take a Welch PSD, fit a straight line to the log-frequency axis, and see whether you get -17 .

MATLAB / Octave

```
w = noisegen([2^18, 4], 48e3);  
[P, f] = pwelch(w(:,1), 8192, 4096, [], 48e3);  
m = f > 60 & f < 0.95*24e3;  
p = polyfit(log10(f(m)), 10*log10(P(m)), 1);  
fprintf('slope = %.1f dB/decade\n', p(1));
```

Python

```
w = noisegen((2**18, 4), 48e3)  
f, P = sg.welch(w[:,0], fs=48e3, nperseg=8192)  
m = (f > 60) & (f < 0.95*24e3)  
s = np.polyfit(np.log10(f[m]), 10*np.log10(P[m]), 1)[0]  
print(f"slope = {s:.1f} dB/decade")
```

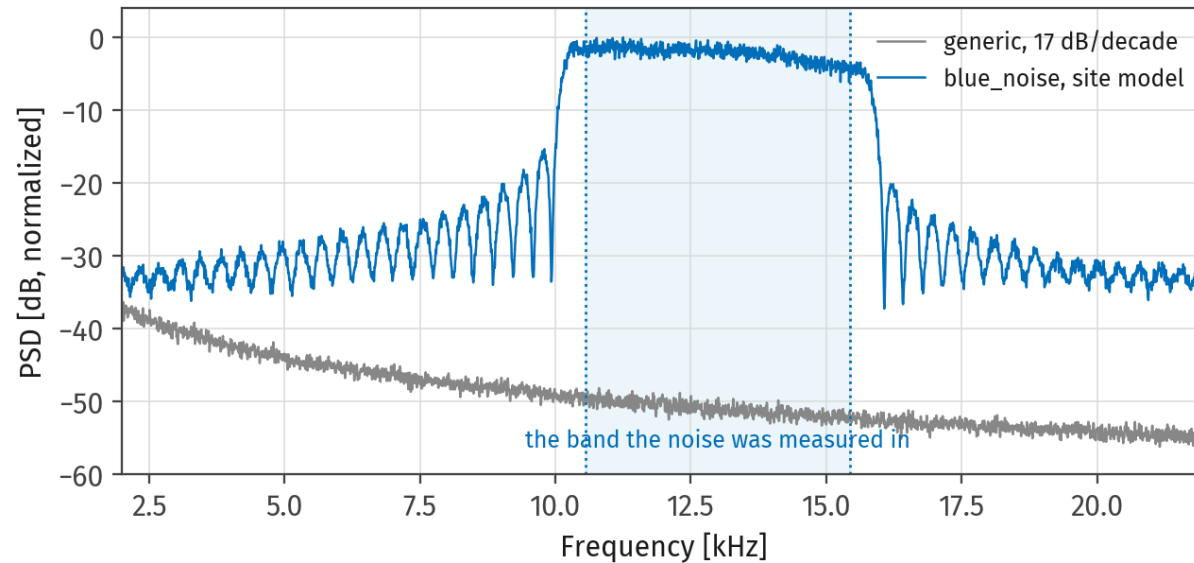
What you should get

slope = -16.9 dB/decade. Not exactly -17 : the model is built as a 4096-point filter, so the fit is a measurement of a real filter rather than of the formula that generated it.

Configuration 2 — the site model



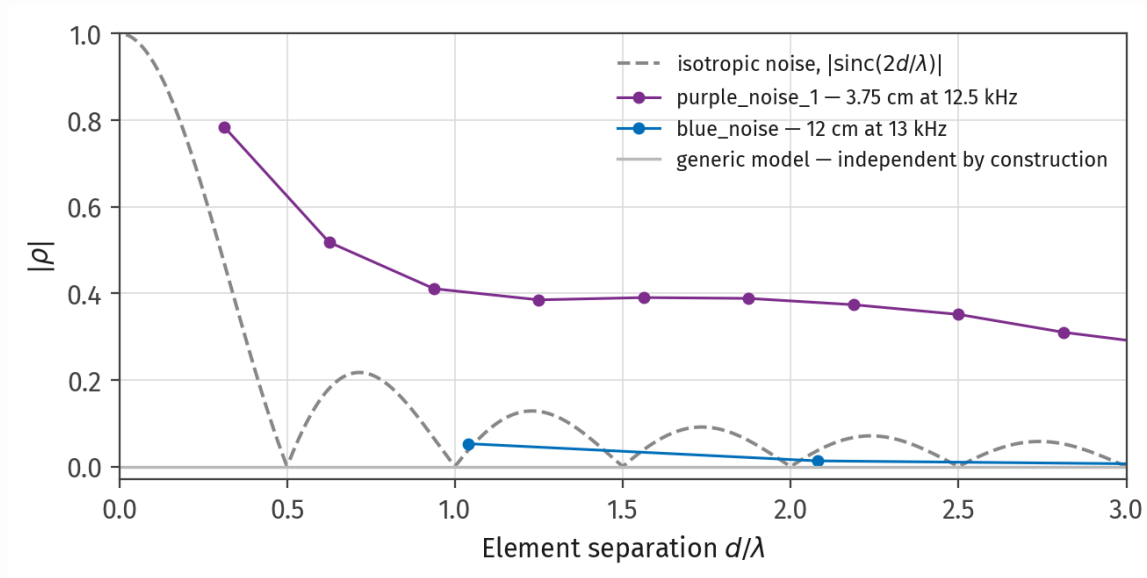
The measured spectrum is not a straight line



blue_noise against the generic model, both normalized to their own peak.

The site model is **band-limited**, because the ambient recording was filtered to the band the channel was measured in before the β were fitted. Outside that band it has no business predicting anything, and it does not pretend to.

And it is correlated across the array



Coherence between elements, measured on noise generated from the released β .

Spacing in **wavelengths** is what matters. **purple** at 3.75 cm and 12.5 kHz sits at 0.31λ and its elements see nearly the same noise. **blue** at 12 cm and 13 kHz sits at 1.04λ and is almost white.

Diagnostic 2 — is your noise correlated?

Two minutes

Generate noise on **all** elements, take the correlation matrix, and look at the first off-diagonal. Then do the same for the generic model.

MATLAB / Octave

```
noise = load('blue_noise.mat');
w = noisegen([2^17, 12], 48e3, 1:12, noise);
C = abs(corrcoef(w));
fprintf('site |rho| = %.3f\n', mean(diag(C,1)));
g = noisegen([2^17, 12], 48e3);
fprintf('generic|rho| = %.3f\n', ...
        mean(diag(abs(corrcoef(g)),1)));
```

Python

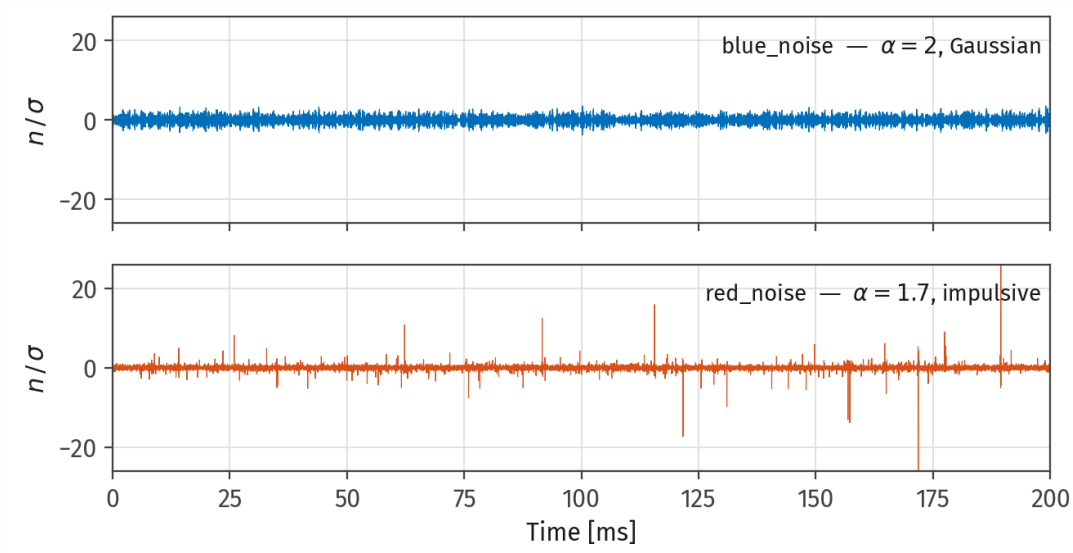
```
noise = load_noise("blue_noise.mat")
w = noisegen((2**17, 12), 48e3, range(12), noise)
C = np.abs(np.corrcoef(w.T))
print(f"site |rho| = {np.diag(C,1).mean():.3f}")
g = noisegen((2**17, 12), 48e3)
Cg = np.abs(np.corrcoef(g.T))
print(f"generic |rho| = {np.diag(Cg,1).mean():.3f}")
```

What you should get

About **0.05** for **blue** and about **0.78** if you repeat it with purple_noise_1.mat over its 32 elements. The generic model gives you whatever your sample size can resolve from zero — it draws every element independently.

Configuration 3 — impulsive noise

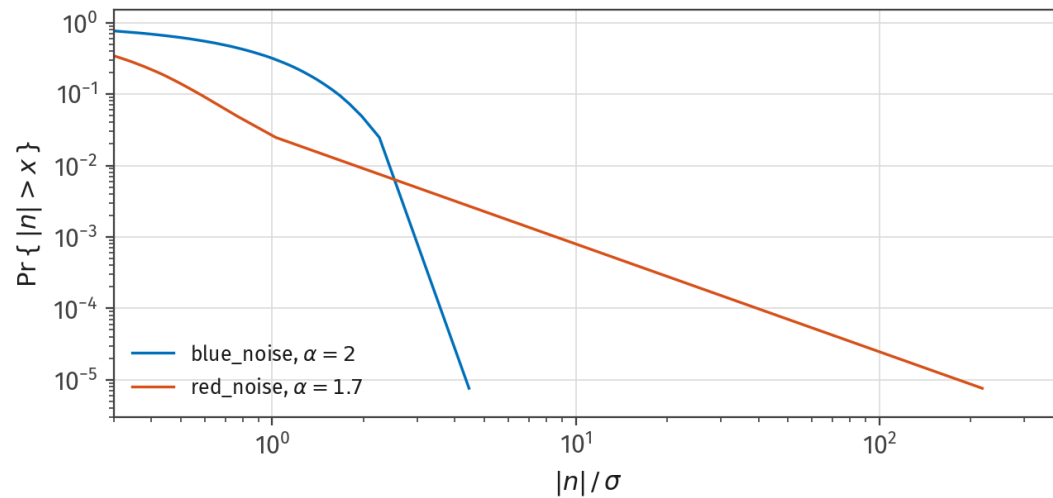
red is the only one, and it looks nothing like the others



200 ms of each, both divided by their own standard deviation. Same vertical scale.

Same σ . Same axis. The Gaussian trace never leaves $\pm 5\sigma$; the impulsive one leaves the frame repeatedly. Snapping shrimp, at roughly a few hundred per second in warm shallow water.

The tail is the whole story



Complementary CDF of $|n|/\sigma$, measured on generated noise.

Quantile	$\alpha = 2$	$\alpha = 1.7$
99%	2.6σ	1.6σ
99.99%	3.9σ	24σ
peak / rms	≈ 5	≈ 250

Note the crossing. The impulsive noise is **quieter** than Gaussian most of the time — and then, rarely, enormously louder. A receiver tuned on the variance is tuned on the wrong statistic.

Diagnostic 3 — find the tail

Two minutes

Generate **red** noise, normalize by its standard deviation, and print the 99.99th percentile. Compare with **blue**.

MATLAB / Octave

```
noise = load('red_noise.mat');
w = noisegen([2^18, 3], 96e3, 1:3, noise);
x = abs(w(:,1)) / std(w(:,1));
fprintf('99.99%: %.1f sigma, peak %.0f sigma\n', ...
        quantile(x, 0.9999), max(x));
```

Python

```
noise = load_noise("red_noise.mat")
w = noisegen((2**18, 3), 96e3, [0,1,2], noise)
x = np.abs(w[:,0]) / np.std(w[:,0])
print(f"99.99%: {np.quantile(x,0.9999):.1f} sigma,"
      f" peak {x.max():.0f} sigma")
```

load_noise, not h5py.File

The MAT-files in circulation are not all saved the same way, and h5py reads only the -v7.3 flavor. load_noise accepts either and hands noisegen exactly what it wants; load_channel does the same for replay and unpack.

Using it properly



Setting the SNR

noisegen gives you a **shape**, not a level. The level is yours, and it is the only number in your simulation that decides what the results mean.

MATLAB / Octave

```
w = noisegen(size(y), fs, array_index, noise);  
r = y + 0.05 * w;
```

Python

```
w = noisegen(y.shape, fs, array_index, noise)  
r = y + 0.05 * w
```

For red, think twice about what “SNR” means

An α -stable process with $\alpha < 2$ has **infinite variance**. The sample variance of any finite record is finite, so the formula above runs and returns a number — but that number depends on how long your record is. Report the recipe you used, or use a dispersion measure instead of a variance.

Six things worth knowing

1. **array_index must match replay.** The noise is correlated across elements; asking for a different set breaks the correspondence between signal and noise.
2. **noisegen resamples for you.** The statistics were measured at `noise.Fs`; you pass **your** `fs`.
3. **The noise is band-limited to the measurement band.** If your signal is inside the channel's band, so is the noise. That is the point.
4. **R in a noise file is a bandwidth**, not a symbol rate. It is the one field name in the library that reads wrong.
5. **One noise file can serve many channels.** All five **purple** noise files apply to all three of its arrays; one `blue_noise` serves all twenty **blue** files.
6. **brown has no site model.** Use the generic one and say so.

1. One function, three configurations: generic, site-specific Gaussian, site-specific impulsive. The last two are the same call — α in the file decides.
2. One equation does all the work: mix white draws across elements and lags with β .
3. **Colored**: the measured spectrum, band-limited to where it was measured.
4. **Correlated**: how much depends on your element spacing in wavelengths, not in meters.
5. **Not always Gaussian**: **red** reaches 24σ where a Gaussian reaches 3.9σ .
6. Set the level from a target SNR. For **red**, say what you meant by SNR.

Section IV of the paper, [arXiv:2609.03207](https://arxiv.org/abs/2609.03207), has the estimation procedure for α and β and the validation against the recordings.

Module 3 — Unpack and visualize

Now open the channel itself, and look at it.

Module 3 — Unpack and Visualize

A Hands-On Tutorial on the Underwater Acoustic Channel Library

Zhengnan Li · Milica Stojanovic

Monday, September 21, 2026

OCEANS 2026 · Monterey Conference Center, Monterey, CA

10:50 – 11:10 · Module 3 of 5

Where we are

09:30 – 10:10	Module 1 — Replay
10:30 – 10:50	Module 2 — Ocean noise
10:50 – 11:10	Module 3 — Unpack and visualize
11:10 – 11:40	Module 4 — The Library at a Glance
11:40 – 12:20	Module 5 — The estimate repository
12:20 – 12:30	Wrap-up and Q&A

The third function, and the one you need least often

`replay` drives a signal through the channel without ever showing it to you. `unpack` is for the times you want to **look** — or to run your own processing on the impulse response rather than on a received waveform.

Why the file is not the channel

A 52-second **blue** recording, sampled at the delay rate, would be a matrix with about half a million rows. The library does not store that. It stores two things:

h_hat

the drift-free response

Motion has been taken **out**, so what remains changes slowly — a few hertz of Doppler spread. It can be sampled in time at $R/100$ and still be complete.

phi_hat

the phase

One number per element per delay-domain sample. It changes fast, so it is kept at the full rate — but it is a **vector**, not a matrix.

A slowly-varying matrix plus a fast-varying vector. That is the whole compression idea, and `unpack` is how you undo it.

MATLAB / Octave

```
h = unpack(fs_time, array_index, channel);
```

Python

```
h = unpack(fs_time, array_index, channel)
```

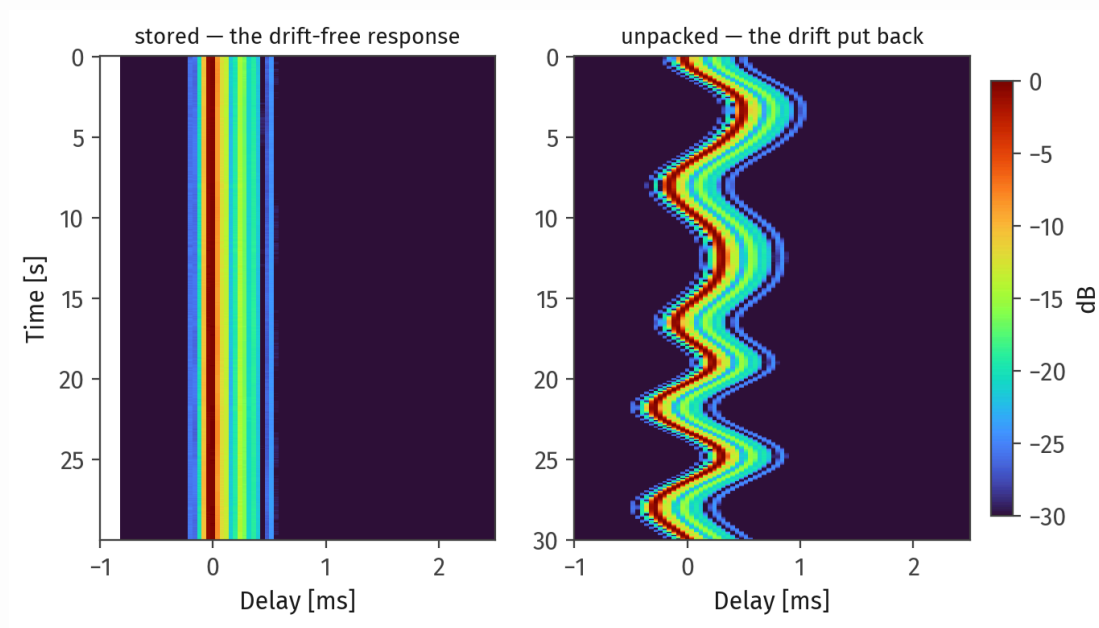
Argument	What it is
<code>fs_time</code>	The rate you want along the time axis [Hz]. Your choice, up to <code>params.fs_delay</code> .
<code>array_index</code>	Which elements. 1-based in MATLAB, 0-based in Python.
<code>channel</code>	The loaded file, exactly as for <code>replay</code> .

Returns a $K \times M \times T$ array: delay taps, elements, snapshots. Three steps happen inside — resample in time, multiply by $e^{j\hat{\phi}}$, then re-insert the delay drift by interpolation. With `theta_hat` the third step is skipped, because the drift never left the taps.

What unpack puts back



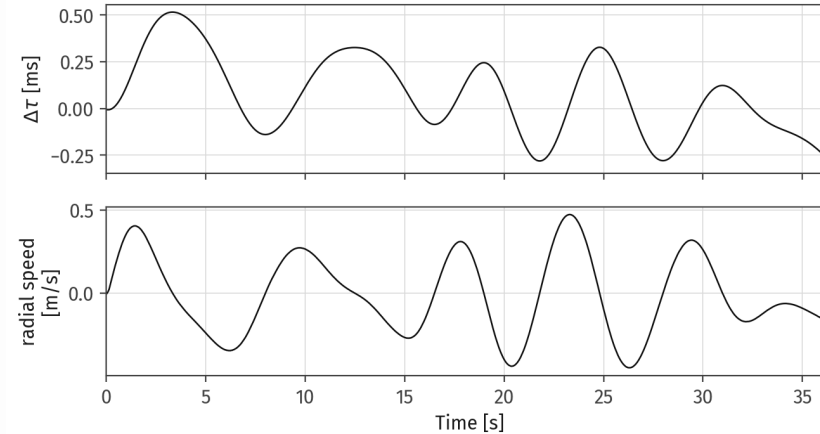
The drift, made visible



black, element 0, first 30 s. Left: the stored tensor. Right: `unpack(25, [0], channel)`.

The same data, before and after. On the left the arrival is a straight vertical line, because that is what compresses. On the right it breathes — and that breathing is real.

It is the ship, on the swell



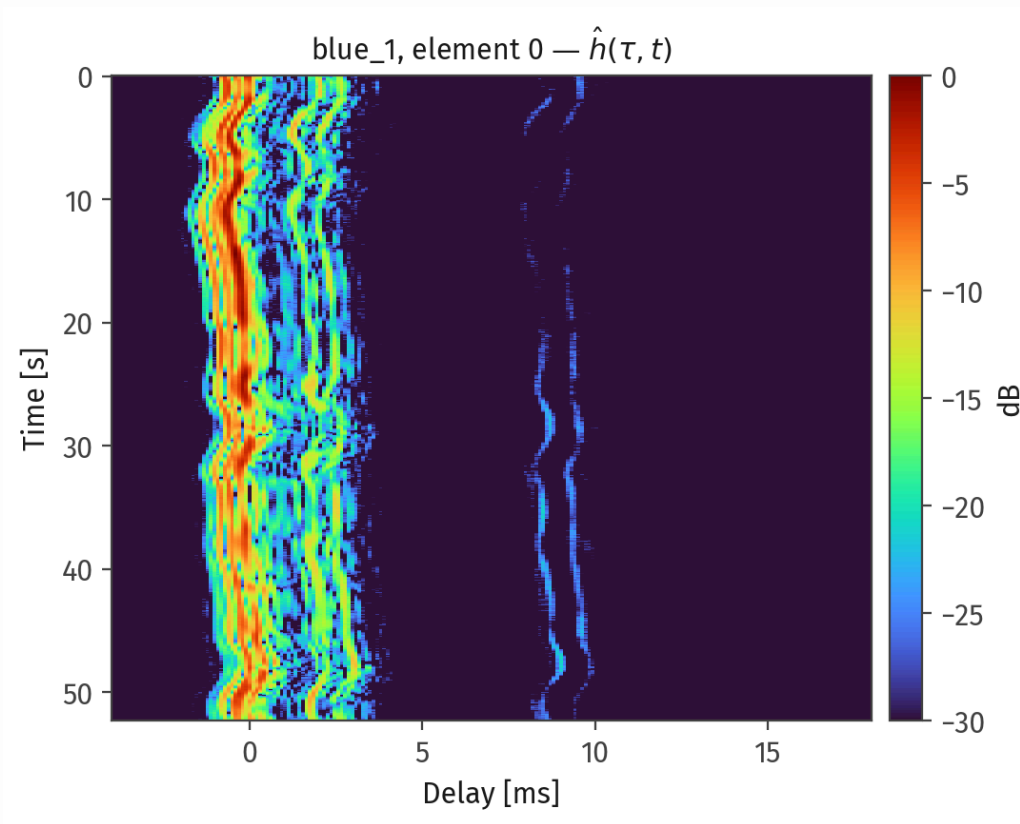
`black`'s `phi_hat`, converted to delay and to radial speed.

$\Delta\tau = -\hat{\phi}/2\pi f_c$, and the speed is its derivative times the sound speed. **0.82 ms peak to peak, ± 0.47 m/s, on a 7.3-second period.** The array hangs 6 m under a ship in the open Pacific — that is the swell.

Worth saying out loud

Nobody put ocean swell into this model. It was measured, stored as a phase vector, and handed back by `unpack`.

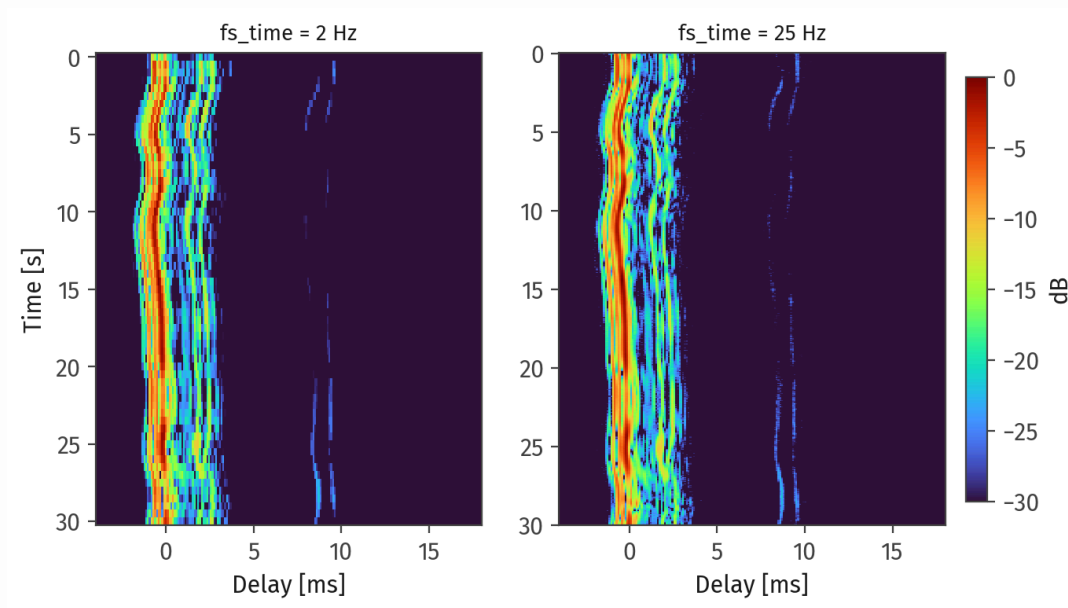
Reading a waterfall



blue, element 0, the whole 52 s at $f_{s_time} = 25$ Hz.

- **Delay runs across, time runs down.** Each horizontal line is one impulse response.
- **Delay is relative to the synchronization point** — the reference sits at $\kappa_{-1} * nsd$ taps in, which `meta` tells you. Energy before zero is not acausal, it is just earlier than the reference.
- **Vertical stripes** mean the geometry is holding still. **Wandering stripes** mean it is not.
- The arrival near 9 ms wanders while the main cluster does not: a surface bounce moving with the sea state.

You choose the time rate



The same 30 s of **blue**, unpacked at two different f_{s_time} .

2 Hz is enough to see the structure and costs a twelfth of the memory. Ask for more than the channel's own Doppler spread and you are interpolating, not measuring — for **blue** the paper puts that spread inside ± 1 Hz.

What it costs

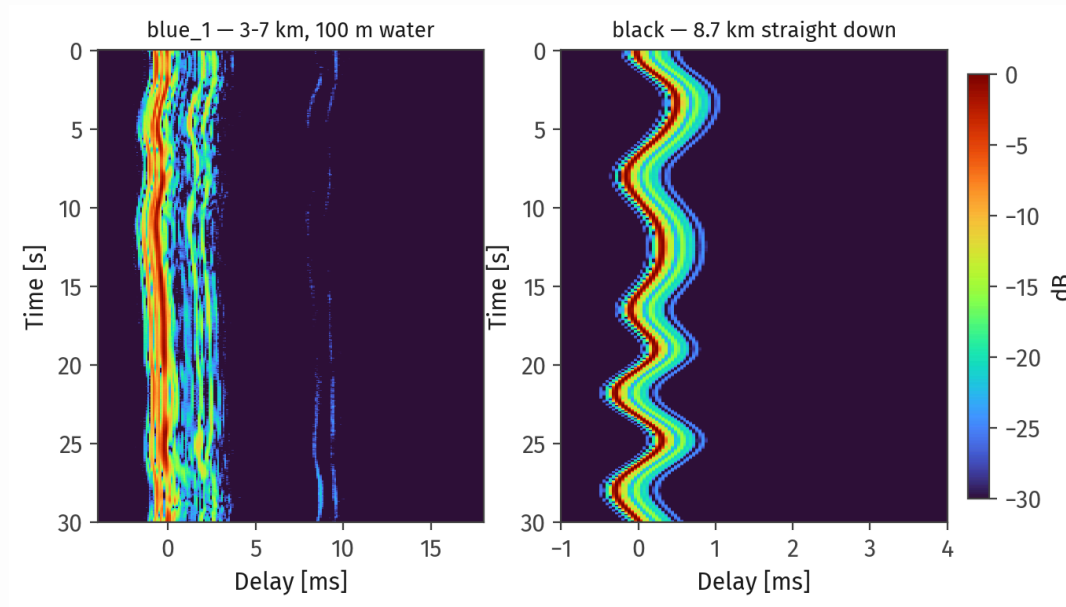
	Size
blue_1, all 12 elements	
Stored: h_hat + phi_hat	206 MB
unpack(2, 1:12, ...)	7.7 MB
unpack(25, 1:12, ...)	96 MB
unpack(fs_delay, 1:12, ...)	37.7 GB

The last row is why the compression exists. It is also why unpack takes the rate as its **first** argument rather than defaulting to something.

Do not unpack in order to replay

replay consumes the compressed file directly and never builds this array. If you find yourself unpacking and then convolving by hand, you have written a slower replay.

Two channels, one function



Both unpacked at 25 Hz, first 30 s, one element each. Note the delay axes.

blue spreads energy over 5 ms with a late arrival at 9 ms. **black** is one arrival, under a millisecond wide, swaying with the swell. Module 1 showed this through a probe; this is the channel itself.

Your turn



Five minutes — unpack and look

1. Unpack `blue_1` on one element at `fs_time = 25` and plot it, delay across and time down, clipped at -30 dB.
2. Find the zero-delay tap: `meta.K_1 * meta.nsd`, plus unpack's left padding.
3. Unpack the same channel at 2 Hz and at 25 Hz. What did you lose, what did you save?
4. **Time left?** Plot `phi_hat`, divide by $-2\pi f_c$, and read off the motion.

MATLAB / Octave

```
channel = load('blue_1.mat');
h = unpack(25, 1, channel);
fsd = channel.params.fs_delay;
k0 = channel.meta.K_1 * channel.meta.nsd;
tau = ((0:size(h,1)-1) - k0 - ceil(0.1*size(channel.h_hat,1)))/
fsd*1e3;
imagesc(tau, (0:size(h,3)-1)/25, ...
        squeeze(20*log10(abs(h(:,1,:)))));
clim([-30 0]), xlabel('Delay [ms]'), ylabel('Time [s]')
```

Python

```
channel = load_channel("blue_1.mat")
h = unpack(25, [0], channel)
fsd = channel["params"]["fs_delay"][0,0]
k0 = (channel["meta"]["K_1"][0,0]*channel["meta"]["nsd"][0,0]
      + np.ceil(0.1*channel["h_hat"].shape[2]))
tau = (np.arange(h.shape[0]) - k0)/fsd*1e3
t = np.arange(h.shape[2])/25
plt.pcolormesh(tau, t, 20*np.log10(np.abs(h[:,0,:])).T,
               vmin=-30, vmax=0)
plt.gca().invert_yaxis(); plt.show()
```

Four ways this goes wrong

1. **Unpacking at `fs_delay` “to be safe”.** 37.7 GB, and no more information than 2 Hz gave you. The stored rate already exceeds the channel’s Doppler spread.
2. **Forgetting where zero delay is.** The reference is `K_1 * nsd` taps in, from `meta`, plus `unpack`’s left padding. Plot without it and every arrival looks late.
3. **Expecting `theta_hat` channels to breathe.** **yellow** and **purple** store the **drifting** response, so `unpack` has nothing to re-insert — the drift was never removed. A flat waterfall there means the file is fine, not that the sea was.
4. **Reading the color scale as absolute.** It is normalized. Use `meta.limit` as the floor; the library plots at -30 dB, and **black** at -50 .

1. `unpack(fs_time, array_index, channel)` → a $K \times M \times T$ array of complex impulse responses.
2. It resamples in time, puts the phase back, and re-inserts the delay drift — the three steps that invert the compression.
3. **You** choose the time rate. 2 Hz is usually enough; `fs_delay` is never necessary.
4. The drift is not a modeling artifact. On **black** it is a ship riding a 7.3-second swell.
5. For driving a signal through the channel, use `replay`. `unpack` is for looking, and for processing you want to do on $\hat{h}(\tau, t)$ itself.

Module 4 — The Library at a Glance

Seven more channels, and how to choose the one you need.

Module 4 — The Library at a Glance

A Hands-On Tutorial on the Underwater Acoustic Channel Library

Zhengnan Li · Milica Stojanovic

Monday, September 21, 2026

OCEANS 2026 · Monterey Conference Center, Monterey, CA

11:10 – 11:40 · Module 4 of 5

Where we are

08:30 – 09:30	Introduction
09:30 – 10:10	Module 1 — Replay
10:10 – 10:30	Coffee
10:30 – 10:50	Module 2 — Ocean noise
10:50 – 11:10	Module 3 — Unpack and visualize
11:10 – 11:40	Module 4 — The Library at a Glance
11:40 – 12:20	Module 5 — The estimate repository
12:20 – 12:30	Wrap-up, contribution workflow, Q&A

Three functions and one channel are behind you. This module is the map: the other seven.

What you should be able to do by 11:40

1. Name all eight channels and say, for each, **what makes it different from the other seven**.
2. Read the parameter table and tell whether a channel **can carry the signal you have in mind**.
3. Choose a channel deliberately, rather than taking the first one in the list.
4. Find, unaided:
 - the **documentation** — what a channel is and what is inside a file,
 - the **data** — the Zenodo record and its DOI,
 - the **code** — matlab, python, estimate on GitHub.
5. Get any of the other seven channels onto your laptop, and know what it costs in disk.

The eight channels



The library in one slide

Eight color-labeled channels

51 channel files + **16** noise files

24.0 GB on Zenodo, CC-BY 4.0

7 repositories on GitHub

Recorded between **November 1994** and
November 2024 by seven groups on four
continents.

Center frequency **75 Hz → 25 kHz**

Bandwidth **37.5 Hz → 12.5 kHz**

Range **60 m → 3250 km**

Water depth **8 m → 8720 m**

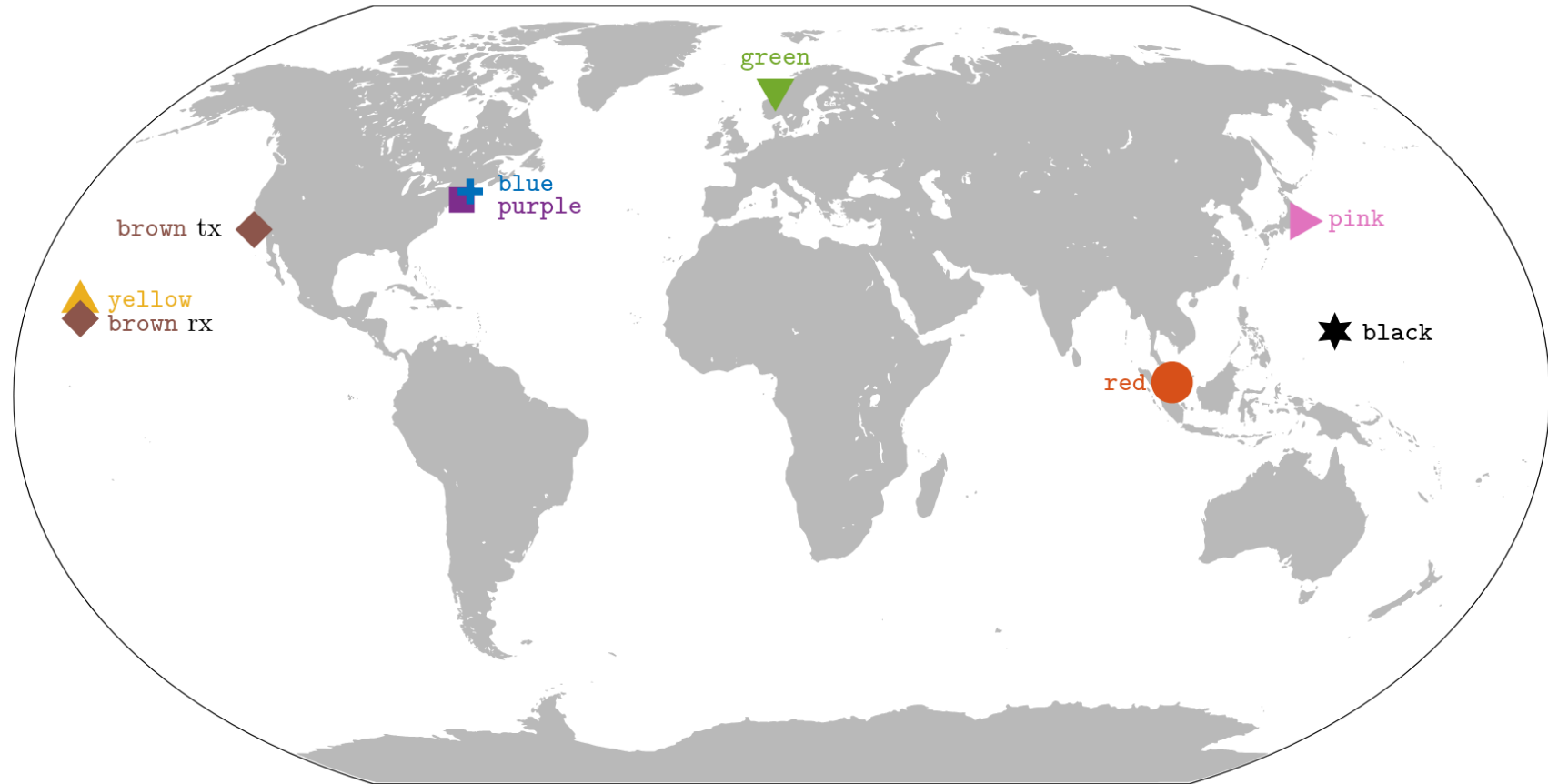
Delay coverage **7 ms → 7 s**

Receivers **1 → 32 elements**

The point of the collection

These are not eight samples of one population. They are eight **different physical situations**, each chosen because it does something the others do not.

Where they were recorded



brown appears twice — its transmitter sat off San Diego and its receiver east of Hawaii, 3250 km apart.

What all eight have in common

- **One probe type.** A pseudo-noise or m-sequence, BPSK or QPSK modulated, repeated, on a single carrier at the center of the band.
- **One estimator.** Adaptive (LMS / RLS / SFTF) with a second-order PLL and, where needed, delay tracking — the estimate repository, Module 5.
- **One storage format.** $\hat{h}$ of size [delay, receiver, time], a phase vector, and a params struct, in HDF5 (v7.3), format version 1.0.
- **Three operations.** replay, noisegen, unpack — identical calls in MATLAB and Python.
- **One restriction.** The channel was observed **only inside the experiment's band**, so it is only valid inside that band.

Parameters (1 / 2)

Codename	Location	Date	$d_T/d_R/d_w$ [m]	Mobility	d [km]	f_c [kHz]	R [ksps]	Array	M	ℓ [m]
blue	North Atlantic	Jun. 2010	30–60/50/100	Mobile	3–7	13	$10^4/2048$	Vertical	12	0.12
red	Singapore	Nov. 2024	6/4.6/8–20	Drifting	0.1–0.4	25	9.6	Vertical	3	0.8
yellow	Hawaii	Jul. 2011	50/50/100	Moored	3	13	6.25	Vertical	24	0.05
					7				24	0.2
					3				16	3.75
					7				16	3.75
purple	North Atlantic	Oct. 2008	11/10/15	Moored	0.06	12.5	$10^4/1536$	Cross	32	0.0375
					0.2			Vertical	24	0.05
					1			Vertical	12	0.12

d_T, d_R, d_w : transmitter, receiver and water depth. d : transmitter–receiver distance. f_c : center frequency. R : probe symbol rate. M : number of array elements. ℓ : inter-element spacing.

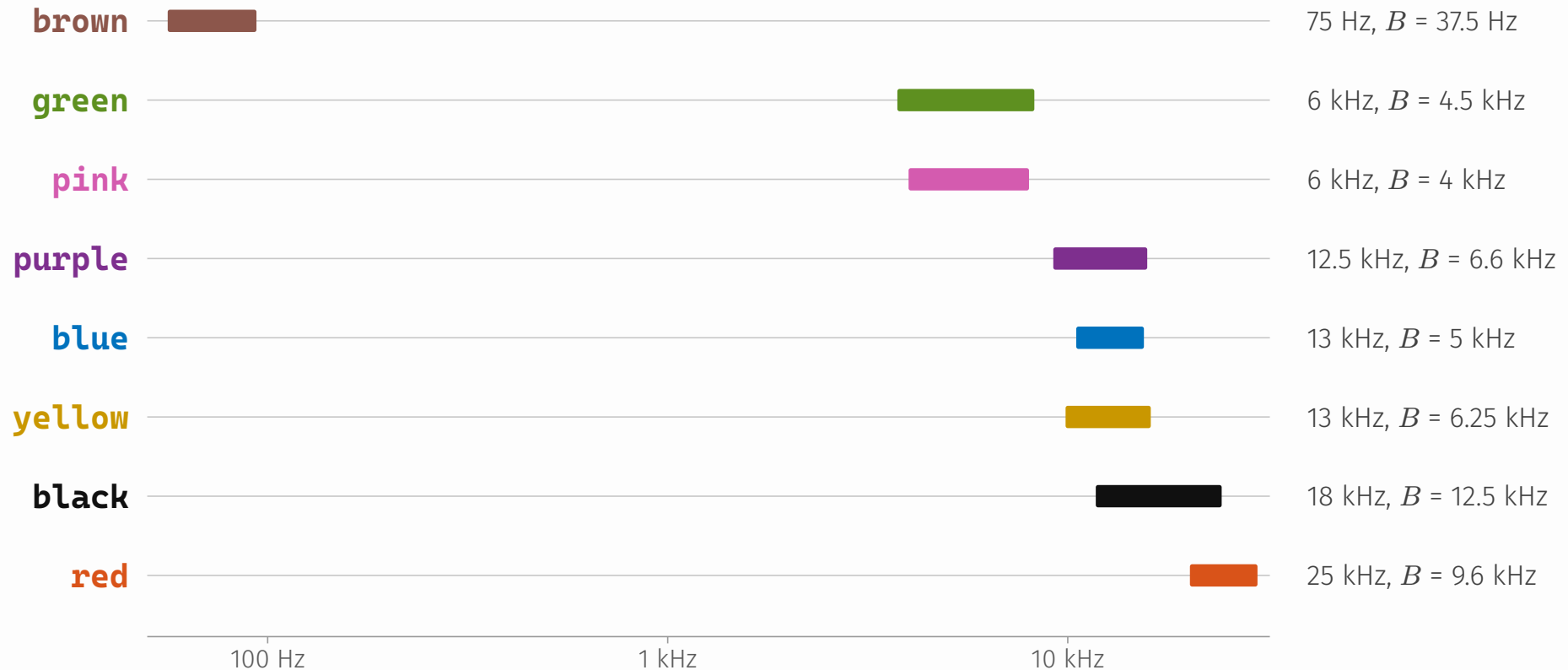
Parameters (2 / 2)

Codename	Location	Date	$d_T/d_R/d_w$ [m]	Mobility	d [km]	f_c [kHz]	R [ksym/s]	Array	M	ℓ [m]
green	Norway	Nov. 2024	20/43/60	Moored	0.27	6	4.5	Time	64	10 min
black	Mariana Trench	Oct. 2024	8718/6/8720	Moored	8.72	18	12.5	Planar	8	≥ 0.088
pink	Japan	Jul. 2022	176/146/x	Moored	14	6	4	Vertical	24	0.9–1.8
brown	Pacific	Nov. 1994	652/900–1600/4000, 5300	Mobile	3250	0.075	0.0375	Vertical	20	35

Two rows that do not mean what they look like

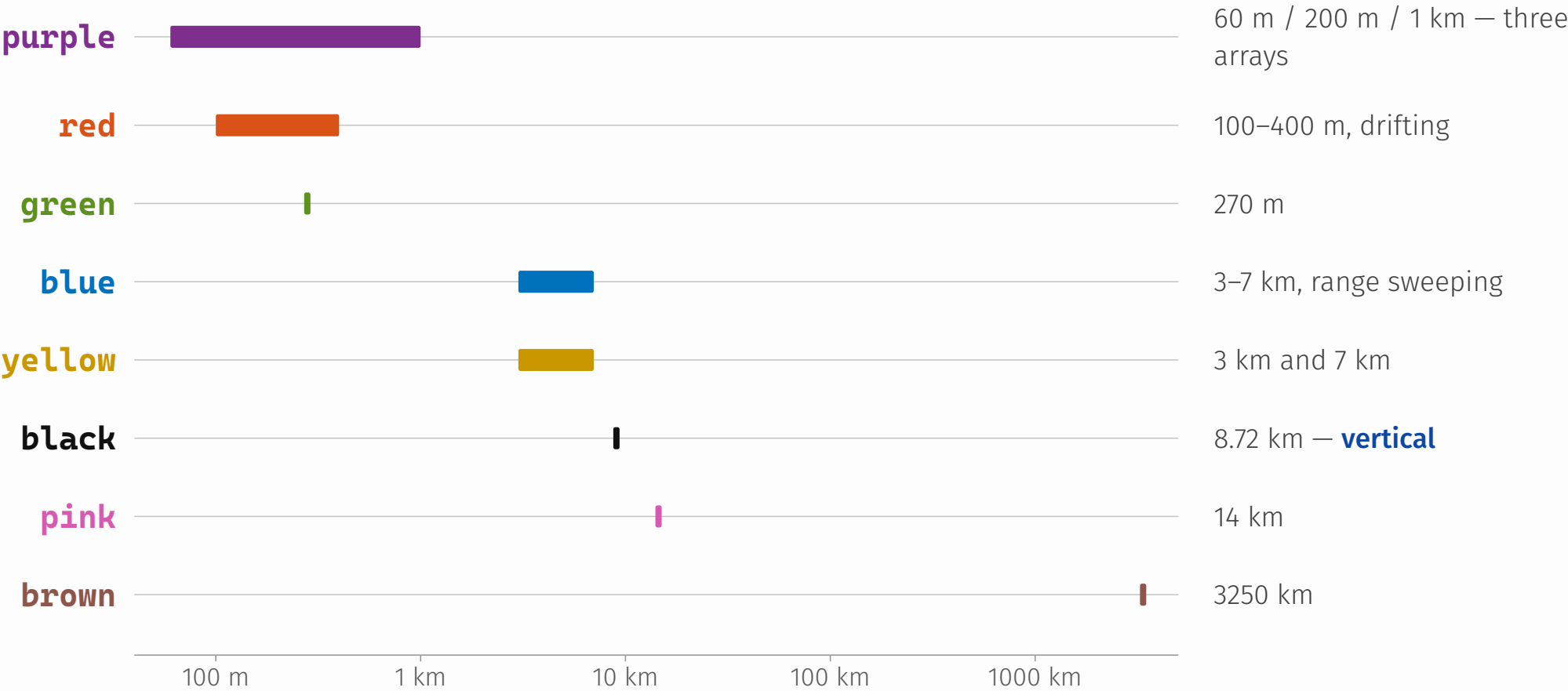
For **green**, M is **not** an array: 64 time-diversity channels from one hydrophone, and ℓ is an interval, not a distance. For **brown**, the two water depths are the ones at the transmitter and at the receiver, 3250 km apart. An “x” was not recorded.

Coverage: the band each channel occupies



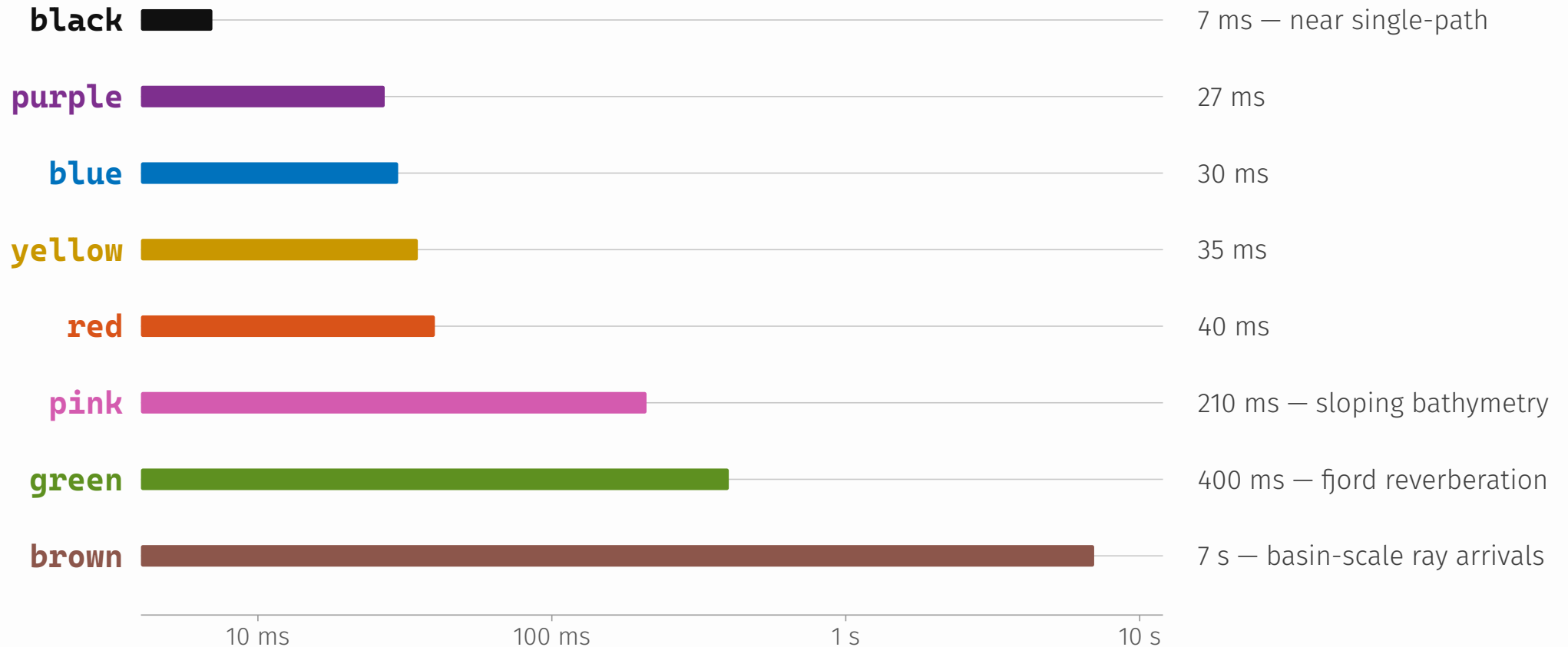
Two and a half decades of carrier frequency. **Your signal has to fit inside one of these bars** — that constraint bites before any other.

Coverage: transmitter–receiver range



Almost five decades. **black** is the odd one — 8.72 km of it is **straight down**, not across.

Coverage: delay spread the file must hold

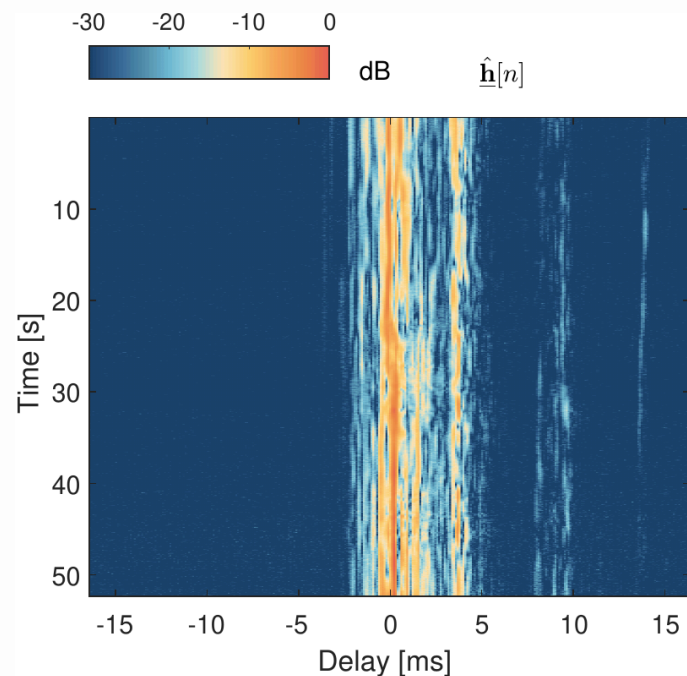


Three decades of stored delay. This is what sets your equalizer length — and the file size.

Coverage: platform, aperture, record length

Codename	Platform	Receiver	Water depth	Time rate	Record
blue	Mobile, ≤ 1.5 m/s	Vertical, 12 el.	100 m	$R/100$	60 s
red	Drifting boat	Vertical, 3 el.	8–20 m	$R/100$	50 s
yellow	Moored	Vertical, 24 el. and 16 el.	100 m	$R/100$	53 s
purple	Moored	Cross 32 el.; vertical 24 and 12 el.	15 m	$R/100$	55 s
green	Moored	Single hydrophone $\times 64$ repeats	60 m	$R/1000$	≈ 55 s
black	Moored	Planar, 8 el., two concentric squares	8720 m	$R/100$	36 s
pink	Moored	Vertical, up to 24 el., 6 groups	≈ 200 m, sloping	$R/100$	55 s
brown	Mobile	Vertical, 20 el., 35 m spacing	4000 / 5300 m	$R/10$	13 min

All eight are sampled in delay at $f_s = 2R$. The **response** is stored coarsely in time ($R/100$, or slower) because it changes slowly; the **phase/delay track** is stored at the full $2R$, because it does not.

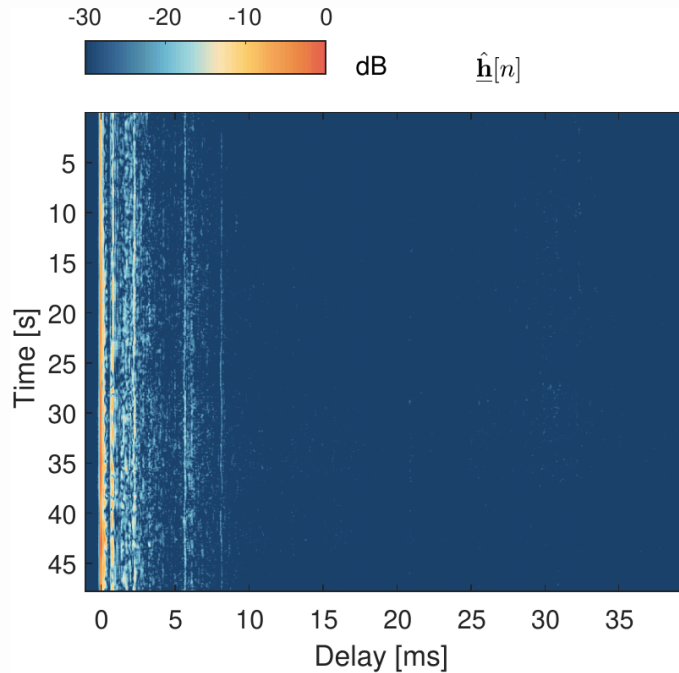


The mobile one. The transmitter was deployed from a vessel under way, circling at up to 1.5 m/s, sometimes closing and sometimes opening; the range swept between 3 and 7 km in 100 m of water. A moored 12-element vertical array at 50 m depth received.

13 kHz carrier, 5 kHz band (10.5–15.5 kHz), BPSK on a 2047-symbol PN sequence.

In the figure: dense multipath within ± 5 ms and two late arrivals near 8 and 14 ms that wander as the geometry changes. The delay drift has already been removed — this is $\hat{h}[n]$, and the drift lives in ϕ_{hat} .

Reach for it when: you are testing Doppler tracking, resampling, or an equalizer that has to survive platform motion. 20 files — the largest ensemble in the library.



The impulsive one. A drifting boat at 100–400 m range in 8–20 m of warm, shallow, biologically loud water, next to a heavy shipping channel. Three hydrophones, 0.8 m apart.

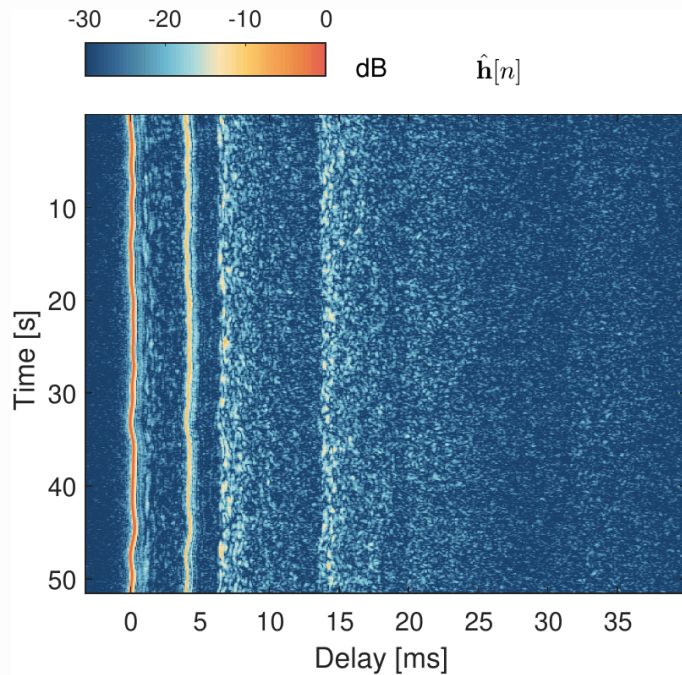
25 kHz carrier — the highest in the library — 9.6 kHz band, BPSK on a generalized m-sequence of length 8191.

In the figure: everything arrives inside 8 ms; a strong direct path and a compact cluster behind it. The channel is the easy part.

The reason this channel exists

Snapping shrimp. **red** carries the **only α -stable noise model** in the library ($\alpha < 2$); every other channel is paired with colored Gaussian noise.

Reach for it when: your receiver assumes Gaussian noise and you want to find out what that assumption costs.

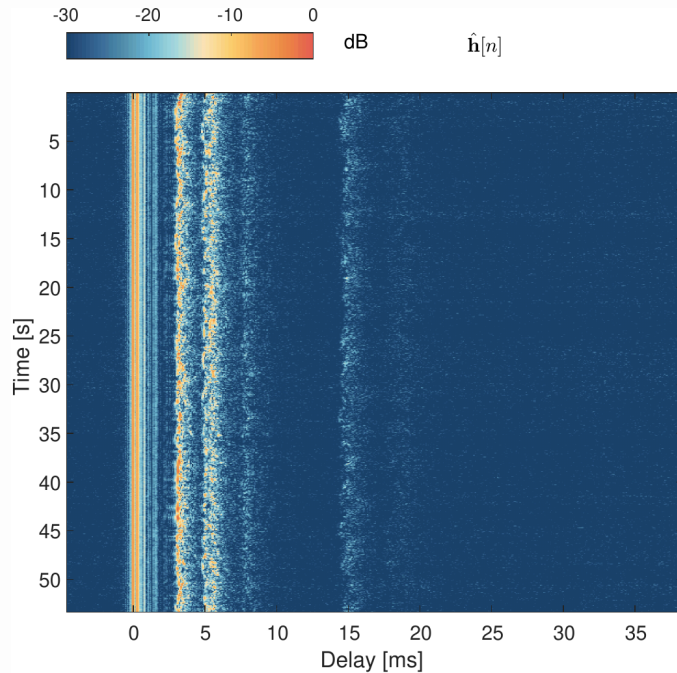


The scattering one. Moored on both ends, 3 km and 7 km, mid-water in a 100 m column off the western side of Kauai. Four receiving systems: 24 elements at 5 cm and at 20 cm, and 16 elements at 3.75 m — apertures from 1.2 m to 60 m on the same link.

13 kHz carrier, 6.25 kHz band, BPSK on a 4095-symbol PN sequence.

In the figure: two crisp early arrivals that meander slowly in delay, then a diffuse scattering field — the “dust” the KAM'11 campaign was built to study — filling everything out past 20 ms. Stored with `theta_hat`: the drift is still in the taps.

Reach for it when: you want array processing with a real choice of aperture, or performance limited by scattering rather than by geometry.

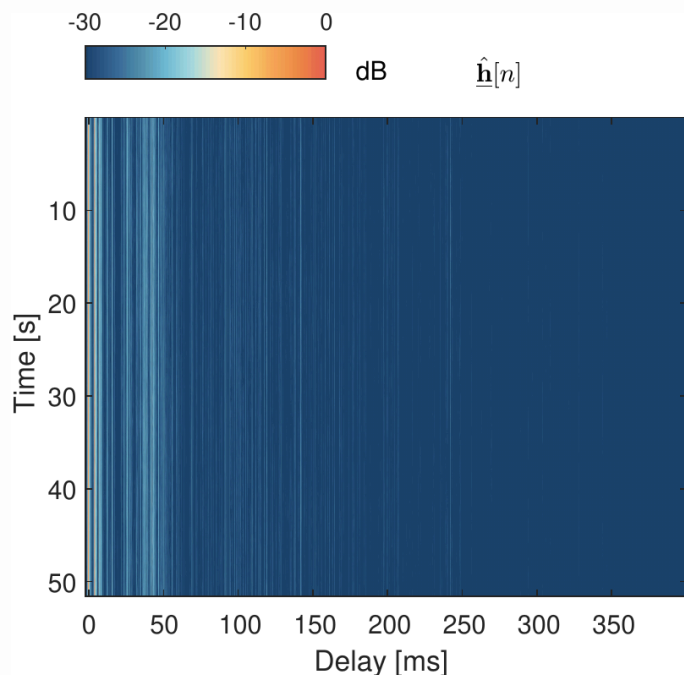


The shallow, surface-driven one. 15 m of water, transmitter at 11 m, receivers at 10 m, and three ranges on the same site: 60 m, 200 m and 1 km. The 60 m station used a **cross array** — two 16-element legs, 32 elements, the only genuine 2-D aperture in the library apart from **black**.

12.5 kHz carrier, 6.6 kHz band, BPSK on a 4095-symbol PN sequence.

In the figure: a hard direct arrival, then a surface/bottom cluster at 3–8 ms that boils from one snapshot to the next as the sea surface moves, and a late bounce near 15 ms. Wind, not range, sets the difficulty here.

Reach for it when: you are benchmarking an equalizer against fast surface fading, or doing spatial processing on a 2-D aperture. 15 files, 5 noise files.



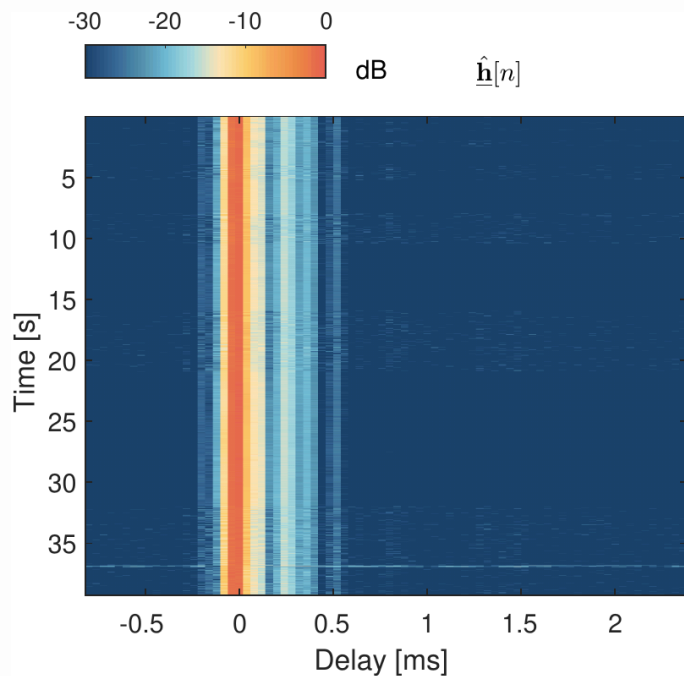
The reverberant one. An archipelago on the west coast of Norway. 270 m of range, 60 m of water — and 400 ms of stored delay. Steep rock walls return energy for a quarter of a second after the direct path.

6 kHz carrier, 4.5 kHz band. One hydrophone, but the transmission was repeated every 10 minutes, giving **64 independent realizations** of the same link.

In the figure: the axis runs to 400 ms, not 40. Energy never really stops; it decays into a floor that stretches across the whole window.

Reach for it when: you need a genuinely long delay spread, or want time diversity — 64 draws of one channel, hours apart — for coding and outage studies.

Contributed by SFI Smart Ocean; also published on IEEE DataPort.

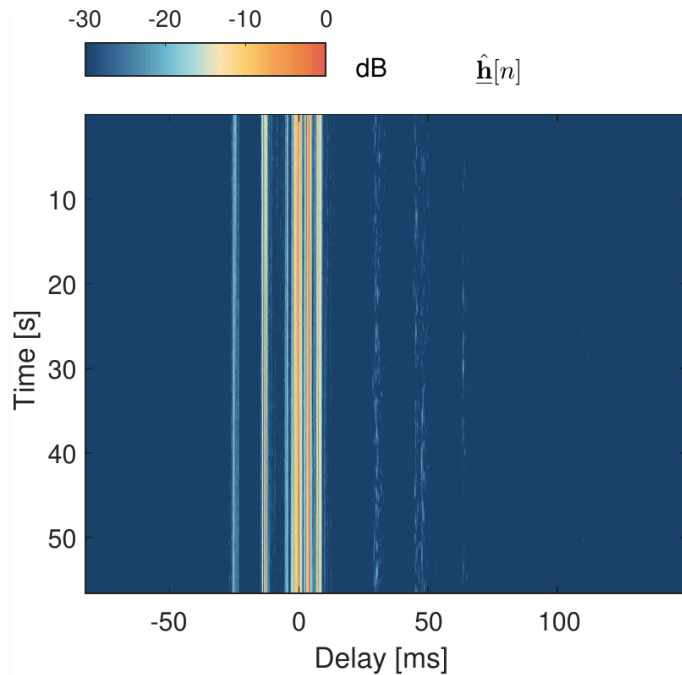


The vertical one. A transmitter 8718 m down, a ship-mounted planar array 6 m below the surface, and 8720 m of water between them. Eight elements on the vertices of two concentric squares (264 mm and 88 mm on a side).

18 kHz carrier, 12.5 kHz band — the **widest** in the library — QPSK on an 8191-symbol PN sequence.

In the figure: note the axis. Everything is inside **1 ms**. A near-vertical path through deep water has almost nothing to bounce off; this is the closest the library comes to an AWGN channel with a real recording behind it.

Reach for it when: you want a benign reference to isolate one effect, a wideband channel, or a deep vertical link.

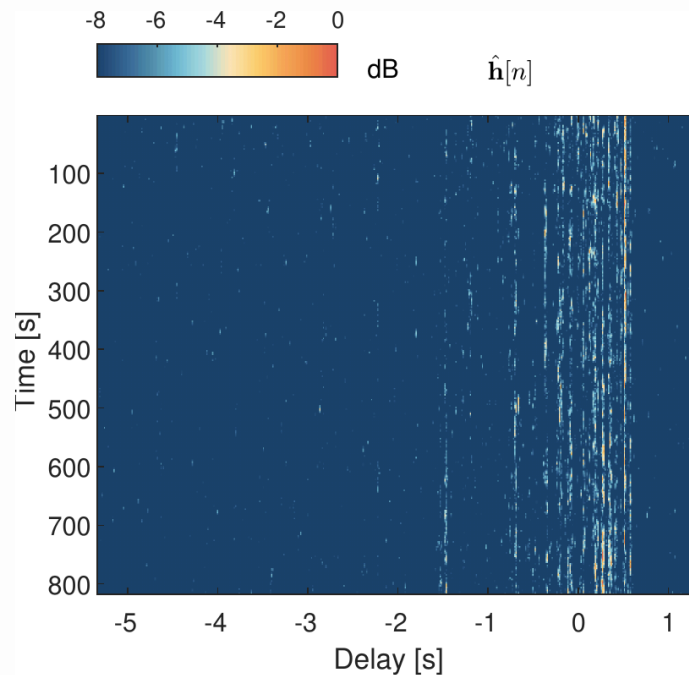


The bathymetry one. 14 km at 6 kHz over a nominally 200 m bottom — recorded three times, over **flat**, **up-slope** and **down-slope** bathymetry. Same link, same hardware, three sea floors: pink_1, pink_2, pink_3.

4 kHz band, QPSK on an 8191-symbol PN sequence. A 24-element vertical array in six groups of four (0.9 m within a group, 1.8 m between groups).

In the figure: arrivals spread from -25 ms to $+65$ ms around the reference, in discrete, stable bundles — refracted ray groups rather than a diffuse cloud.

Reach for it when: you want to change the bottom geometry and hold everything else fixed — a controlled comparison you rarely get from at-sea data.



The basin-scale one. A 75 Hz source at 652 m off San Diego, a 20-element autonomous vertical array east of Hawaii spanning 900–1600 m depth, and **3250 km** of Pacific in between. Elements 35 m apart — a 665 m aperture.

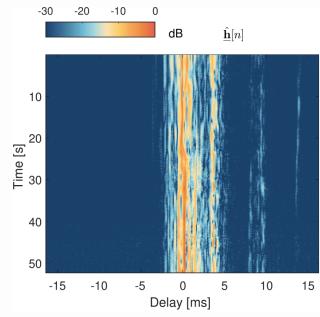
37.5 Hz of bandwidth, 37.5 sym/s, BPSK on a 1023-symbol PN sequence.

In the figure: the delay axis is in **seconds** and the time axis in **minutes**. Sparse, sharply-defined ray arrivals, non-minimum phase, spread over roughly two seconds.

Reach for it when: you work at very long range and very narrow band — or when you want to remind an audience that “the underwater channel” covers five orders of magnitude in range.

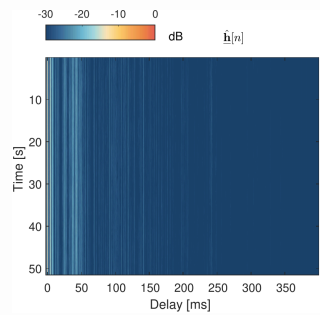
Acoustic Engineering Test of the ATOC project; Worcester et al., JASA 105, 3185 (1999).
The only channel with **no** site-specific noise model.

All eight, same axes of thought



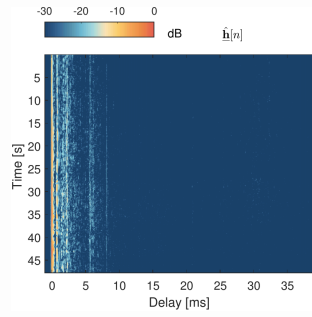
blue

mobile, 3–7 km



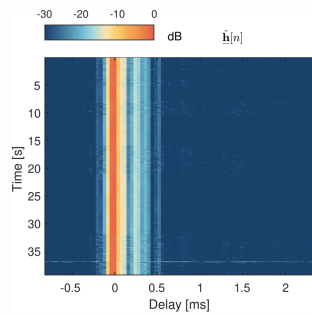
green

400 ms fjord tail



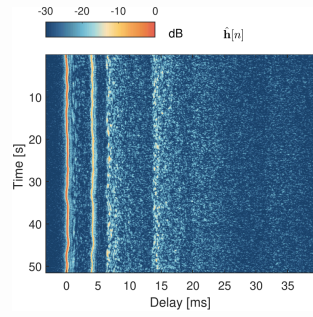
red

impulsive noise



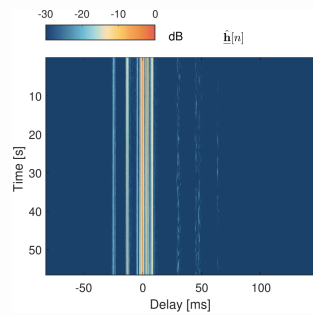
black

< 1 ms, vertical



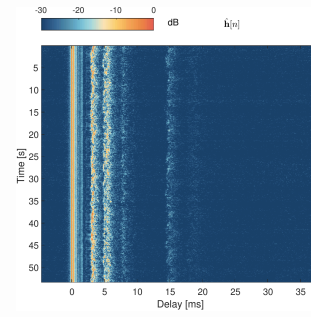
yellow

scattering “dust”



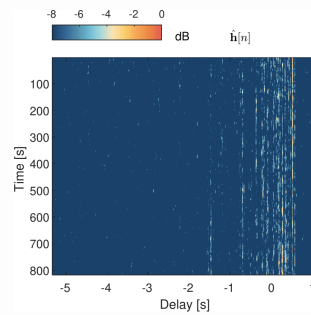
pink

sloping bottom



purple

surface bounces



brown

3250 km, 75 Hz

One receiver element from each channel. Delay horizontal, time vertical, magnitude in dB — but **look at the axis limits**, they differ by three decades.

Choosing a channel

There is no best channel

The honest criterion is not quality. All eight were estimated with the same pipeline and all eight are good estimates.

The criterion is **distinctness**. Each channel was brought into the library because it puts a different stress on a receiver:

blue — the transmitter is moving

red — the noise is not Gaussian

yellow — the volume scatters

purple — the surface moves fast

green — the reverberation will not die

black — there is barely a channel at all

pink — the sea floor tilts

brown — the range is continental

So the question to ask is not “which is best?”

It is: **which physical effect do I want my receiver to confront, and which hard limits can my signal live inside?**

Choose by the effect you want to stress

I want to test a receiver against...	Pick	Because
Doppler and platform motion	blue	Transmitter under way at up to 1.5 m/s, range sweeping 3–7 km
Impulsive, non-Gaussian noise	red	The only α -stable noise model in the library
Fast surface fading in shallow water	purple	15 m of water, wind-driven surface bounces at 3–8 ms
Diffuse volume scattering	yellow	The KAM'11 “dust” — a scattering field out past 20 ms
A very long delay spread	green	400 ms of fjord reverberation at only 270 m range
Bathymetry, everything else held fixed	pink	Flat, up-slope and down-slope over the same 14 km link
A nearly benign channel	black	Under 1 ms of spread on a vertical path
Extreme range and a very narrow band	brown	3250 km at 75 Hz, 37.5 Hz of bandwidth

Choose by what your algorithm needs

My algorithm needs...	Pick	Note
A large array	purple (32, cross)	Only 2-D aperture besides black
	yellow (24 / 16)	Four apertures, 1.2 m to 60 m
	pink (24), brown (20)	665 m aperture on brown
Many realizations for statistics	blue (20 files)	Largest ensemble
	purple (15 files)	Three ranges × five recordings
Time diversity across hours	green (64 repeats)	10 minutes apart — not an array
Wide bandwidth	black (12.5 kHz)	Then red (9.6 kHz)
A long observation window	brown (13 min)	Everything else is 36–60 s
To run fast, on a laptop	black (162 MB)	pink is 1.5 GB per file

The constraint that bites first: bandwidth

From the User's Guide

“The channels are specified for a certain acoustic bandwidth that was used during the experiment. When working with a channel, note that **only that bandwidth is captured**. If you are designing a signal to pass through a channel, the bandwidth of your signal must fit within the stated limit.”

The channel was probed with a signal of bandwidth B centered at f_c . Outside $[f_c - \frac{B}{2}, f_c + \frac{B}{2}]$ nothing was measured, so nothing is modeled. A wider signal does not get a worse answer — it gets a meaningless one.

Widest: **black** 12.5 kHz, **red** 9.6 kHz

Typical: **purple** 6.6, **yellow** 6.25, **blue** 5 kHz

Narrow: **green** 4.5, **pink** 4 kHz

Narrowest: **brown** 37.5 Hz

Three other limits, in the order people hit them:

1. **Elements.** array_index cannot exceed M . **red** has 3.
2. **Duration.** You get 36–60 s of channel (**brown**: 13 min).
3. **Size.** **pink** is 1.5 GB per file; **black** is 162 MB.

Every channel comes with its noise

Codename	Channel files	Noise file(s)
blue	blue_1 ... blue_20	blue_noise
red	red_1 ... red_4	red_noise
yellow	yellow_1 ... yellow_6	yellow_noise_1 ... _4
purple	purple_1 ... purple_15	purple_noise_1 ... _5
green	green	green_noise
black	black	black_noise
pink	pink_1 ... pink_3	pink_noise_1 ... _3
brown	brown	<i>none</i> — use the generic model

Each model is fitted to the recording itself and stored as a characteristic exponent α and a set of mixing coefficients $\beta_{ij}(kT_s)$, which together carry the spectrum **and** the spatial correlation across the array:

$$\hat{n}_i(nT_s) = \sum_{j=0}^{M-1} \sum_{k=0}^L \beta_{ij}(kT_s) \eta_j(nT_s - kT_s)$$

$\alpha = 2$ is Gaussian. Only **red** has $\alpha < 2$.

A **generic** model is also provided: log-linear decay with frequency, nominally 17 dB/decade, uncorrelated in space, with user-set power. That is what **brown** uses.

The five **purple** noise files apply to all three of its arrays. This table, the two model families and the generic model are all on the site's **Noise** tab, /noises/ — and Module 2 was built on it.

Where everything lives



Three places, three jobs

Documentation

`uwa-channels.github.io`

What each channel **is**.

What is **inside a file**. How to install, replay, unpack.

Free, fast, no login. Start here.

Data

`doi:10.5281/
zenodo.21287414`

The channels themselves: 67 files, 24.0 GB, CC-BY 4.0, versioned and citable.

Download only what you need.

Code

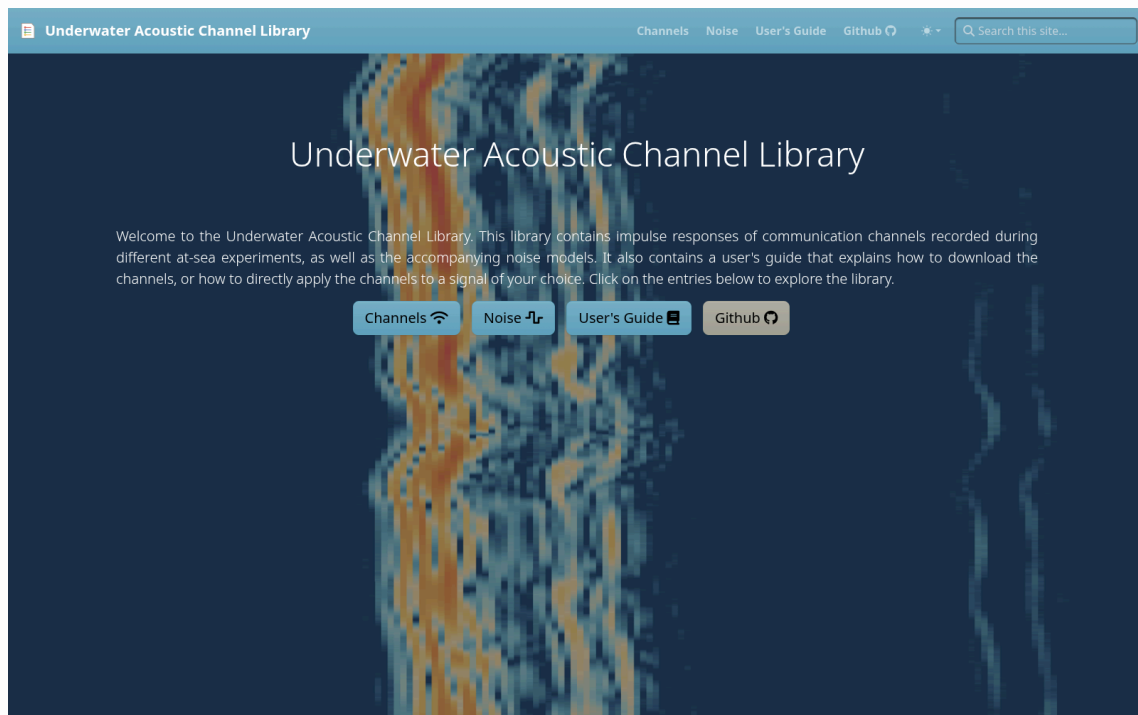
`github.com/uwa-channels`

matlab, python — replay, noisegen, unpack.
estimate — how the channels were made.

Issues, discussions, pull requests.

The three are deliberately separate: **documentation changes weekly, code changes monthly, data is archived and immutable.**

The documentation site



`uwa-channels.github.io`

Four entry points, and that is the whole site:

- **Channels** — the master table, the map, one page per channel
- **Noise** — the models and the channel-to-noise pairing
- **User's Guide** — install, replay, unpack, file format
- **Github** — out to the code

Plus a **search box** (offline index, so it works in a conference room) and a light/dark toggle.

Channels tab — the table you will come back to

Underwater Acoustic Channel Library

ChannelsNoiseUser's GuideGithub

Search this site...

Search this site...

Channels

Blue

Red

Yellow

Purple

Green

Black

Pink

Brown

Channels

Eight color-labeled underwater acoustic channels recorded across sites worldwide, spanning shallow and deep water, short and long range, and fixed and mobile platforms.

The channels contained in this library come from various experiments. The experiments differ by geographical location, transmission distance, water depth, transmitter/receiver mobility, acoustic bandwidth, and the size of the recording array. The channels are color-labeled, and a detailed description of relevant parameters is given below for each. The channels are available for download from Zenodo:

DOI

10.5281/zenodo.21287414

(https://doi.org/10.5281/zenodo.21287414)

Each channel is estimated from the experimental data and stored as a tensor of complex-baseband impulse responses evolving over time, with dimensions [delay, receiver, time]. If a recording array is available, the tensor holds one matrix for each array element. Each column of this matrix is an instantaneous channel response as a function of delay, that is, the channel vector $\mathbf{h}[n]$ at time index n . Successive columns correspond to different, equi-spaced instants in time. Any motion-induced delay drift is suppressed in the channel matrix, leaving the drift-free response $\hat{\mathbf{h}}(\tau, t)$, which compresses efficiently for storage. A separate vector, the phase estimate $\hat{\varphi}(nT_s)$ stored as `phi_hat`, is provided that contains the uncompressed time-varying channel phase, which is directly related to the delay drift. Here T_s is the sampling interval along the delay axis. To fully reconstruct the channel, the stored matrix needs to be decompressed, and the phase/delay needs to be imparted to re-introduce the phase shift and the delay drift. Details of this process, along with the ready-to-use code, are given in the [User's Guide](#). The stored variable names, and the alternative convention in which only the phase $\hat{\theta}(nT_s)$ is tracked (`theta_hat`) and the delay drift is left in the taps, are documented in the [channel file format specifications](#).

Codename	Location	Date	$d_r/d_w/d_w$ [m]	Mobility	d [km]	f_c [kHz]	R [ksym/s]	Array	M	ℓ [m]
Blue	North Atlantic	Jun. 2010	30-60/50/100	Mobile	3-7	13	10 ⁴ /2048	Vertical	12	0.12
Red	Singapore	Nov. 2024	6/4.6/8-20	Drifting	0.1-0.4	25	9.6	Vertical	3	0.8
Yellow	Hawaii	Jul. 2011	50/50/100	Moored	3	13	6.25	Vertical	24	0.05
					7				24	0.2
					3				16	3.75
					7				16	3.75
Purple	North Atlantic	Oct. 2008	11/10/15	Moored	0.06	12.5	10 ⁴ /1536	Cross	32	0.0375

View Markdown

View page source

Edit this page

Create child page

Create documentation issue

Print entire section

/channels/

The same parameter table you saw a few slides ago, plus:

- the Zenodo DOI badge, linked
- the world map
- a sidebar with the eight channel pages
- a short statement of **what a stored channel actually is** — [delay, receiver, time], drift-free taps, phase kept separately

Each row's codename links to that channel's page.

A channel page — every number in one place

The screenshot shows the 'Underwater Acoustic Channel Library' website. The header includes a search bar and navigation links for 'Channels', 'Noise', 'User's Guide', and 'Github'. A sidebar on the left lists channel categories: Blue, Red, Yellow, Purple, **Green**, Black, Pink, and Brown. The main content area for the 'Green' channel includes a description: 'A 6 kHz single-hydrophone channel from a Norwegian fjord (Norway'24), providing time-diversity data from repeated transmissions over a 270 m range.' Below this is a list of experimental details and parameters. An 'Acknowledgement' block at the bottom states that the dataset was contributed by the SFI Smart Ocean group and provides a link to the IEEE Dataport.

Underwater Acoustic Channel Library

Channels Noise User's Guide Github

Search this site...

Search this site...

Channels

Blue
Red
Yellow
Purple
Green
Black
Pink
Brown

Channels / Green

Green

A 6 kHz single-hydrophone channel from a Norwegian fjord (Norway'24), providing time-diversity data from repeated transmissions over a 270 m range.

- Experiment:
 - Location: 60N, 5.3E
 - Date recorded: November 2024
 - Center frequency: $f_c = 6$ kHz
 - Bandwidth: $B = 4.5$ kHz
 - Symbol rate: $R = 4.5$ ksym/s
 - Distance: $d = 270$ m
 - Tx/Rx/water depth: $d_T/d_R/d_W = 20$ m/43 m/60 m
 - Mobility: moored
 - Receiving array: none (single hydrophone)
 - Time diversity: $M = 64$ channels formed from transmissions repeated every 10 minutes
- Parameters:
 - Sampling rate in delay: $f_s = 2R$
 - Total delay coverage: up to 400 ms
 - Sampling rate in time: $R/1000$ for the channel response, $2R$ for the phase/delay
 - Total time coverage: approximately 55 seconds

Acknowledgement

The Underwater Acoustic Channels Group gratefully acknowledges the contribution of the Green dataset from SFI Smart Ocean. For more information, please visit the [IEEE Dataport link](#).

View Markdown
View page source
Edit this page
Create child page
Create documentation issue
Print entire section

/channels/green/

Every channel page has the same two blocks:

Experiment — location, date, f_c , B , R , d , depths, mobility, receiving array.

Parameters — sampling rate in delay, total delay coverage, sampling rate in time, total time coverage.

Where a dataset was contributed, an **acknowledgment** block names the contributor and links their own release.

The screenshot shows the 'User's Guide' page of the 'Underwater Acoustic Channel Library'. The page has a blue header with navigation links: 'Channels', 'Noise', 'User's Guide' (active), and 'Github'. A search bar is on the right. On the left, there's a sidebar with a search bar and links to 'User's Guide', 'Channel file format specifications', and 'Code Contribution Guide'. The main content area is titled 'User's Guide' and contains instructions for downloading the library, replaying a signal, and visualizing a decompressed channel. It mentions that channels can be used in two ways: applied directly to a user-generated signal or decompressed for visualization. A section titled 'Installation' shows two tabs: 'MATLAB/Octave' (selected) and 'Python'. The MATLAB/Octave tab contains a code block with the command: `matlab.addons.install(websave([tempname '.mltbx'], 'https://github.com/uwa-channels/matlab/releases/latest/download/uwa-channels.mltbx'), 'https://github.com/uwa-channels/matlab/releases/latest/download/uwa-channels.mltbx')`. Below this, it explains that the MATLAB command downloads the latest release and installs it as an add-on, so `replay`, `noisegen`, and `unpack` are on the path. It also provides instructions for installing from a clone and under Octave, including a code block: `git clone https://github.com/uwa-channels/matlab.git` and `cd matlab`. A final code block shows `install`. On the right, there's a sidebar with links like 'View Markdown', 'View page source', 'Edit this page', 'Create child page', 'Create documentation issue', and 'Print entire section'. Below that, it says 'On this page' and lists 'Installation', 'Installing from a clone, and under Octave', 'Requirements', 'Applying a channel to an arbitrary signal', and 'Visualizing a channel'.

/docs/

- **Installation** — one line for MATLAB, one for Python, tabbed side by side
- Installing from a clone, and under Octave
- Requirements: MATLAB R2021a+ with Signal Processing Toolbox; Octave 9.0+ with signal and statistics; Python 3.10+
- **Applying a channel to a signal** — `replay` then `noisegen`
- **Visualizing a channel** — `unpack`

One warning worth reading twice

Respect the bandwidth. And you do **not** need to unpack before replaying.

File format specifications — the authoritative page

/docs/file_specifications/

Required in every channel file

- `h_hat` — complex tensor `[delay, receiver, time]`
- `params` — `fs_delay`, `fs_time`, `fc`
- `version` — currently 1.0

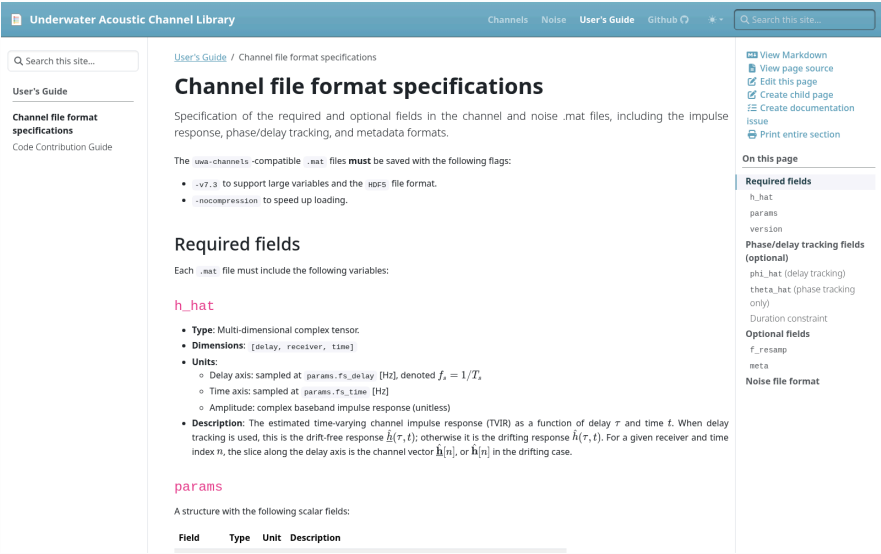
Optional, phase and delay

- `phi_hat` — delay tracking; `h_hat` is drift-free
- `theta_hat` — phase only; drift stays in the taps
- `f_resamp` — a constant resampling factor

Optional, strongly encouraged

- `meta` — codename, filter lengths, optimizer, μ or λ , PLL gains, element spacing, ...

Saved with `-v7.3` (HDF5) and `-nocompression`.



phi_hat or theta_hat — the one distinction to carry forward

phi_hat — delay tracking

h_hat holds the **drift-free** response $\hat{h}(\tau, t)$.
The common delay drift was taken out,
which is why the file compresses well.

$$\hat{\phi}(nT_s) = -2\pi f_c \Delta\tau(nT_s)$$

unpack puts the phase back, then
evaluates the result at shifted instants to
reinstate the drift.

blue red green black pink brown

theta_hat — phase tracking only

h_hat holds the **drifting** response $\hat{h}(\tau, t)$ —
the drift is embedded in the taps. Only
the carrier phase is tracked separately.

$$v(t) = \sum_n d(n) h(t - nT, t) e^{j\theta(t)} + w(t)$$

yellow purple

Precedence: if both are present, phi_hat wins. f_resamp is applied independently of either. If none is present, no Doppler correction is applied.

The Zenodo record

The screenshot shows the Zenodo record page for the 'Underwater Acoustic Channel Repository'. The header includes the Zenodo logo, a search bar, and links for 'Communities' and 'My dashboard'. A notice states: 'Zenodo is currently experiencing slowness and intermittent outages due to heavy automated traffic from bots and AI crawlers. We are aware of the problem, and our team is focused on stabilizing the service. Thank you for your patience.' The repository title is 'Underwater Acoustic Channel Library'. Below the title, it says 'Published August 1, 2026 | Version 1.0'. The authors listed are Li, Zhengnan^{1,2}; Chitre, Mandar³; Cuji, Diego^{4,2}; Preisig, James⁵; Singer, Andrew^{4,6}; Stojanovic, Milica²; and van Walree, Paul⁷. A 'Show affiliations' button is present. The dataset statistics show 464 views and 3K downloads. The 'Versions' section lists 'Version 1.0' with the DOI 10.5281/zenodo.21287414, dated Aug 1, 2026. The 'External resources' section mentions it is indexed in OpenAIRE. The 'Files' section shows a single file named 'Files (24.0 GB)'. A cookie consent banner is at the bottom.

zenodo Search records... Communities My dashboard Log in Sign up

Zenodo is currently experiencing slowness and intermittent outages due to heavy automated traffic from bots and AI crawlers. We are aware of the problem, and our team is focused on stabilizing the service. Thank you for your patience.

Underwater Acoustic Channel Library

Published August 1, 2026 | Version 1.0 Dataset Open

Underwater Acoustic Channel Repository

Li, Zhengnan^{1,2}; Chitre, Mandar³; Cuji, Diego^{4,2}; Preisig, James⁵; Singer, Andrew^{4,6}; Stojanovic, Milica²; van Walree, Paul⁷

Show affiliations

This dataset contains measured underwater acoustic channel impulse responses (CIRs) collected during at-sea experiments spanning diverse geographic locations, environmental conditions, and system configurations. Each CIR captures how a transmitted acoustic signal is modified by the ocean environment as a function of propagation delay, and evolves over time to reflect the non-stationary nature of the underwater channel.

The collection currently includes eight channel datasets, identified by color labels (Blue, Red, Yellow, Purple, Green, Black, Pink, Brown). These span locations including the North Atlantic, Singapore, Hawaii, Norway, Japan, the Mariana Trench, and the Pacific Ocean, with transmission ranges from tens of meters to thousands of kilometers, center frequencies from 75 Hz to 25 kHz, and receiver configurations ranging from single hydrophones to vertical, cross, and planar arrays.

Each channel is stored as a compressed complex-baseband channel matrix in MATLAB format (.mat), accompanied by a phase vector that encodes the time-varying delay drift suppression. Full reconstruction instructions and ready-to-use toolboxes in MATLAB and Python are available at <https://github.com/uwa-channels>.

This library is intended to support reproducible research and benchmarking in underwater acoustic communications, channel modeling, and signal processing. Researchers can apply the provided channels directly to signals of their choice using the companion software.

Files

Files (24.0 GB)

Name	Size
------	------

This site uses cookies. Find out more on how we use cookies

Accept all cookies Accept only essential cookies

464 VIEWS 3K DOWNLOADS Show more details

Versions

Version 1.0 Aug 1, 2026
10.5281/zenodo.21287414

Cite all versions? You can cite all versions by using the DOI 10.5281/zenodo.21287413. This DOI represents all versions, and will always resolve to the latest one. Read more.

External resources

Indexed in OpenAIRE

Communities

Underwater Acoustic Channel Repository, v1.0, August 1, 2026

Li · Chitre · Cuji · Preisig · Singer · Stojanovic · van Walree

- **67 files, 24.0 GB**
- **CC-BY 4.0** — free to use, cite the record
- Indexed in OpenAIRE
- Versioned: a v1.1 would get its own DOI and this one would keep resolving

The data lives here, not on GitHub, because 24 GB does not belong in a git repository.

Two DOIs, and which one to cite

Concept DOI — “the library”

10.5281/zenodo.21287413

Always resolves to the **latest** version.

Cite this when you mean the library as a resource — in a related-work section, an acknowledgment, a README.

Version DOI — “v1.0”

10.5281/zenodo.21287414

Pins **exactly** the files you used.

Cite this in a paper whose results someone should be able to reproduce. This is the one on the documentation site.

Rule of thumb

Reproducibility is the whole point of the library. If your numbers depend on the data, cite the **version** DOI and name the files — purple_2.mat, purple_noise_2.mat.

What is actually in the record

Codename	Files	Each	Noise
blue	20	206 MB	1 file, 0.2 MB
red	4	192 MB	1 file
yellow	6	361–985 MB	4 files
purple	15	354–713 MB	5 files
green	1	1.09 GB	1 file
black	1	162 MB	1 file
pink	3	1.44–1.50 GB	3 files
brown	1	661 MB	—

51 channel files, 16 noise files.

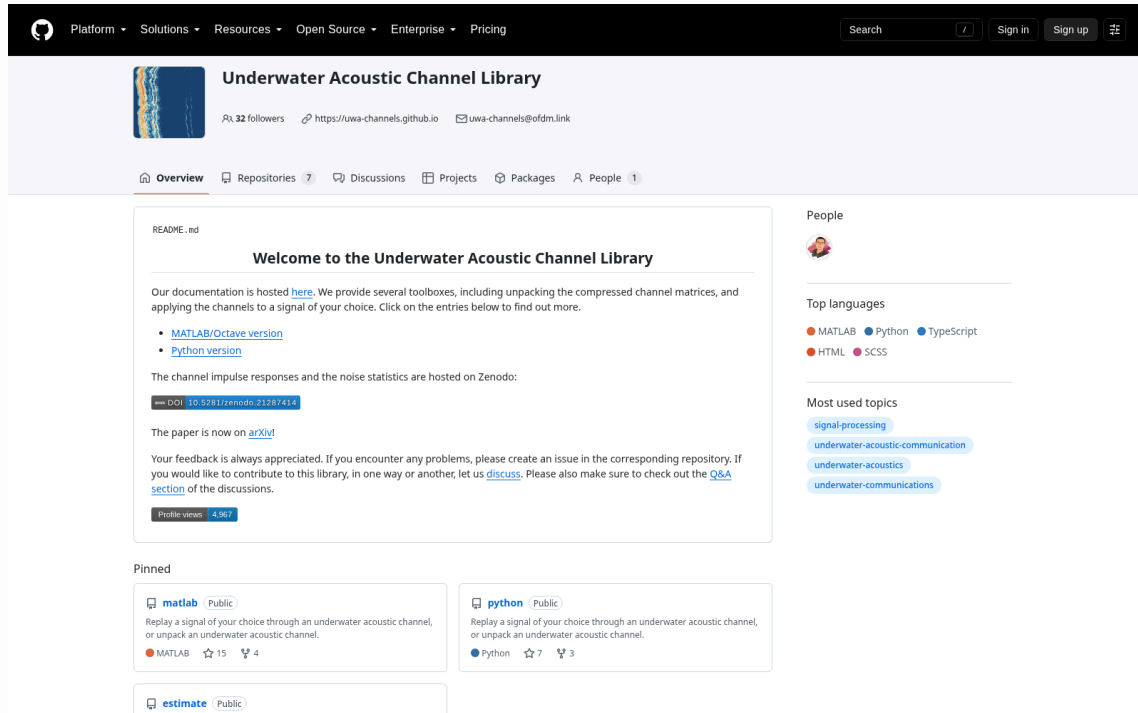
You do not need 24 GB. One channel file plus its noise file is a complete working set — typically 200 MB to 1.5 GB.

Getting the next one

Pick the channel by the effect you want to stress, then download it and its noise file **at home**. Allow 1 GB free.

The room network will not carry a hundred people fetching half a gigabyte each — which is why this morning's two files were announced ahead of time.

The GitHub organization



github.com/uwa-channels — seven public repositories.

Three matter today:

matlab — the MATLAB/Octave toolbox

python — the Python package

estimate — how a channel is made from a recording

The other four:

uwa-channels.github.io — the documentation site;
every page has a **View page source** and an **Edit this page** link, so a typo fix is a two-minute pull request

oceans26-tutorial — this tutorial; slides, templates
and reference solutions are archived here afterwards
browser-replay — replay a signal in a browser, nothing
installed

.github — the organization profile

Where to put what: Issues for a bug, with a minimal
reproducible example and your version numbers.
Discussions for questions and “is this channel right
for my problem?”. **Pull requests** branch from main, add
a test, run CI.

matlab and python — the same three functions

[uwa-channels/matlab](#) — MATLAB R2021a+ with Signal Processing Toolbox, or Octave 9.0+

```
src/      replay.m  noisegen.m  unpack.m
tests/    testReplay testNoise  testUnpack
examples/ example_replay.m
          example_unpack.m
```

Install as an add-on from the packaged .mltbx release, or clone and run `install`. **Not both** — two copies on the path and load order decides.

There is an [Open in MATLAB Online](#) button if your license will not travel.

[uwa-channels/python](#) — Python 3.10 – 3.12

```
src/uwa_channels/  replay  noisegen  unpack
tests/             test_replay.py
                   test_noise.py
                   test_unpack.py
examples/          example_replay.py
                   example_unpack.py
```

`pip install uwa-channels.load_channel / load_noise` open the MAT-files.

Both repositories run CI on every push and carry the same CONTRIBUTING.md.

Each repository carries a CONTRIBUTING.md with the full workflow. Julia support lives outside the organization, in UnderwaterAcoustics.jl, maintained by ARL.

The two APIs, side by side

MATLAB / Octave

```
channel = load('blue_1.mat');  
noise   = load('blue_noise.mat');  
array_index = [1, 2, 3];  
  
y = replay(input, fs, array_index, channel);  
w = noisegen(size(y), fs, array_index,  
noise);  
r = y + 0.05 * w;
```

Python

```
from uwa_channels import (replay, noisegen,  
                           load_channel, load_noise)  
channel = load_channel("blue_1.mat")  
noise   = load_noise("blue_noise.mat")  
array_index = [0, 1, 2]  
y = replay(input, fs, array_index, channel)  
w = noisegen(y.shape, fs, array_index, noise)  
r = y + 0.05 * w
```

Same signature, one difference

array_index is **1-based** in MATLAB and **0-based** in Python. The 0.05 is σ_n , the noise scaling that sets your SNR — the same line you ran in Module 1.

estimate — where channels come from

uwa-channels/estimate — MATLAB/Octave.

Two functions:

- **est.m** — adaptive channel estimation. LMS, RLS or SFTF, a second-order PLL, and optional delay tracking.
- **plots.m** — the visualizations: time-varying IR, Doppler-delay scattering function, delay-angle beamforming, multipath intensity profile, PLL phase.

`main.m` runs a synthetic BPSK example end to end, with known ground truth.

It writes exactly the format the library reads:

`h_hat, phi_hat` OR `theta_hat, params, meta, version`.

Which is the point. **Estimate your own recording with this, and it drops straight into replay** — and, if you want, into the library.

Module 5, 11:40

Bring a recording if you have one. We will make a first estimation pass at it.

Recap

What to take out of this module

1. **Eight channels, each for a reason.** Motion (**blue**), impulsive noise (**red**), scattering (**yellow**), surface fading (**purple**), reverberation (**green**), a near-benign vertical path (**black**), bathymetry (**pink**), continental range (**brown**).
2. **Choose by the effect you want to stress**, then check the hard limits: bandwidth first, then elements, duration and file size.
3. **Three places.** Documentation on `uwa-channels.github.io`, data on Zenodo under `doi:10.5281/zenodo.21287414`, code at `github.com/uwa-channels`.
4. **One distinction to remember.** `phi_hat` means the taps are drift-free; `theta_hat` means the drift is still in them. Check which your file has.
5. **One thing to do next.** Pick the channel that stresses the effect you care about, and pull it and its noise file from Zenodo.

Module 5 — The estimate repository

Where every one of these files came from, and how to make one of your own.

Module 5 — The `estimate` Repository

A Hands-On Tutorial on the Underwater Acoustic Channel Library

Zhengnan Li · Milica Stojanovic

Monday, September 21, 2026

OCEANS 2026 · Monterey Conference Center, Monterey, CA

11:40 – 12:20 · Module 5 of 5 · bring a recording if you have one

Where we are

09:30 – 10:10	Module 1 — Replay
10:30 – 10:50	Module 2 — Ocean noise
10:50 – 11:10	Module 3 — Unpack and visualize
11:10 – 11:40	Module 4 — The Library at a Glance
11:40 – 12:20	Module 5 — The estimate repository
12:20 – 12:30	Wrap-up and Q&A

The module that runs backwards

Everything so far started from a file in the library. This one ends with one. Ten minutes of it are yours, and a live demonstration needs a volunteer with a phone.

The round trip



... and that file is what Modules 1, 2 and 3 consumed.

Three reasons to care:

- You have a recording nobody has modeled, and you want to simulate against it.
- You want to know what the assumptions behind the library's channels actually are.
- You want to contribute a channel, in which case this is the format gate.

What you must have kept

The estimator is **data-aided**. It does not find the channel from the recording alone — it finds it from the recording and the symbols you sent.

You need	Why
d	The transmitted symbols. Not the waveform — the symbols. A seed is enough if the sequence is reproducible.
F_s	The sampling rate of the recording.
R	The symbol rate you transmitted at.
f_c	The carrier you modulated onto.
r	The recording itself, <code>[time, M]</code> , real passband.

The most common reason a sea trial cannot be turned into a channel

Nobody wrote down the symbol sequence. A recording without its d is a recording, not a channel. Seed your generator, and put the seed in the cruise log.

How the estimator works



The model it inverts

$$v(t) = \sum_n d(n) h(t - nT, t) e^{j\theta(t)} + w(t)$$

Three unknowns, deliberately separated:

- $h(\tau, t)$ — the **impulse response**. Many taps, changes slowly.
- $\theta(t)$ — the **carrier phase**. One number per element, changes fast.
- $w(t)$ — noise, which is Module 2's problem, not this one's.

Why split the phase off at all

Because it moves orders of magnitude faster than the rest. Estimating a 320-tap vector quickly enough to follow the phase would need a step size so large that the taps themselves would never settle. Track the phase separately and the filter can be slow, which is what makes it accurate.

Five stages

1 Synchronize

Cross-correlate the baseband recording against the first `sync_length` symbols (255 by default) and find the peaks. `delta` nudges the result by whole samples.

2 Down-shift and resample

Multiply by $e^{-j2\pi f_c t}$, then resample to `nsd` samples per symbol — 2 is enough for a band-limited signal. Normalize the power.

3 Adapt the filter

Slide the known symbols through a $K_1 + K_2$ tap filter and drive the error to zero with LMS, RLS or SFTF. This is $\hat{h}$.

4 Track the phase

A second-order PLL fed by $\psi = -\Im\{\tilde{v} e^*\}$, with gains K_{f_1} (proportional) and K_{f_2} (integral). This is $\hat{\theta}$.

5 Track the delay (optional)

Resample the input on the fly at $t_n = nT_s - \hat{\theta}/2\pi f_c$, so the arrival never walks out of the filter window. This is what turns $\hat{\theta}$ into $\hat{\phi}$.

Why stage 5 exists

Without delay tracking, a moving platform drags the whole arrival structure across the filter window. To keep it, you must make the window longer — and a longer window needs a smaller step size, which makes the estimate slower and blurrier, **and** a wider matrix to store.

Without delay tracking

`theta_hat`

The taps hold the drift. The window must cover **delay spread + total drift**. Fine for a moored deployment; expensive for a mobile one.

With delay tracking

`phi_hat`

The drift is removed before the filter sees it. The window only has to cover the **delay spread**, and the drift is kept as a phase vector.

Set `param.delay_tracking = true` and the saved field is `phi_hat`; leave it false and it is `theta_hat`. That is the distinction Module 4 asked you to carry.

What comes out

```
[h_hat, theta_hat] = est(data, param);  
  
params.fs_delay = param.R * param.nsd;  
params.fs_time  = param.R * param.nst;  
params.fc       = param.fc;  
  
save('mine.mat', 'h_hat', 'phi_hat', ...  
     'params', 'meta', 'version', ...  
     '-v7.3', '-nocompression');
```

The two sampling rates are not free parameters — they fall out of your estimator settings:

- $fs_delay = R \cdot nsd$
- $fs_time = R \cdot nst$

With $nsd = 2$ and $nst = 1/100$, a 4 ksymbols/s probe gives 8 kHz in delay and 40 Hz in time — which is exactly the shape of every file in the library.

meta carries everything needed to reproduce the estimate: κ_1 , κ_2 , the optimizer, its step size or forgetting factor, the PLL gains.

-v7.3 -nocompression, every time

MATLAB's save writes v5 unless told otherwise, and the Python toolbox reads HDF5. Forget the flag and your file is unreadable from Python.

Driving it



The knobs that matter

Knob	Default in <code>main.m</code>	What it decides
κ_1, κ_2	30, 100 symbols	The filter window: anti-causal and causal reach. Must cover your delay spread.
<code>nsd</code>	2	Samples per symbol in delay. 2 suffices for a band-limited signal.
<code>nst</code>	1/100	How often an impulse response is stored. Sets <code>fs_time</code> .
<code>nslr</code>	2	Delay-tracking rate, $T_r = T_s / \text{nslr}$.
<code>optim</code>	1 (LMS)	1 LMS, 2 RLS, 3 SFTF.
<code>mu</code>	$0.3 / (K_1 + K_2) / \text{nsd}$	LMS step size. Scaled by filter length, not chosen freely.
<code>Kf_1, Kf_2</code>	$10^{-3}, 10^{-4}$	PLL proportional and integral gains.
<code>sync_length</code>	255	Symbols used for the correlation template.

κ_2 is the one to get right first. At 8 ksym/s, 100 causal symbols is 12.5 ms of reach enough for **blue**, nowhere near enough for **green**.

Which optimizer?

Optimizer	Setting	When
LMS	$\mu = 0.3/K / n_{sd}$	The default. Cheapest per sample, slowest to converge — the estimator discards the first $20K$ symbols.
RLS	$\lambda = 0.995$	Converges in about $2K$ instead. Costs $O(L^2)$ per sample.
SFTF	$\lambda = 0.999$	RLS convergence at close to LMS cost, at the price of numerical fragility.

Measured on one 8.7 s recording, four elements:

Optimizer	Residual	NumPy	JAX
LMS	−17.5 dB	1.17 s	0.36 s
RLS	−16.7 dB	17.38 s	4.12 s
SFTF	−17.9 dB	4.20 s	0.62 s

Start with LMS. Reach for SFTF when the channel changes faster than LMS can follow.

Designing a probe you can estimate from

- **A pseudo-noise sequence**, BPSK or QPSK, repeated. Flat spectrum, sharp autocorrelation, and reproducible from a seed.
- **Long enough to average, short enough to stay coherent.** The library's channels use periods of 1023 to 8191 symbols.
- **Inside the band your hardware is flat in.** Not the band you wish it had.
- **With guard time.** The tail has to land somewhere, and it makes the burst findable.
- **At a known carrier.** Write down f_c , R , F_s and the seed. All four.

The library's own probes

blue: 2047-symbol PN, BPSK, 13 kHz. **red**: 8191-symbol m-sequence, BPSK, 25 kHz. **black** and **pink**: 8191-symbol PN, QPSK. **brown**: 1023 symbols at 37.5 sym/s — two carrier cycles per chip.

main.m — the check you run first

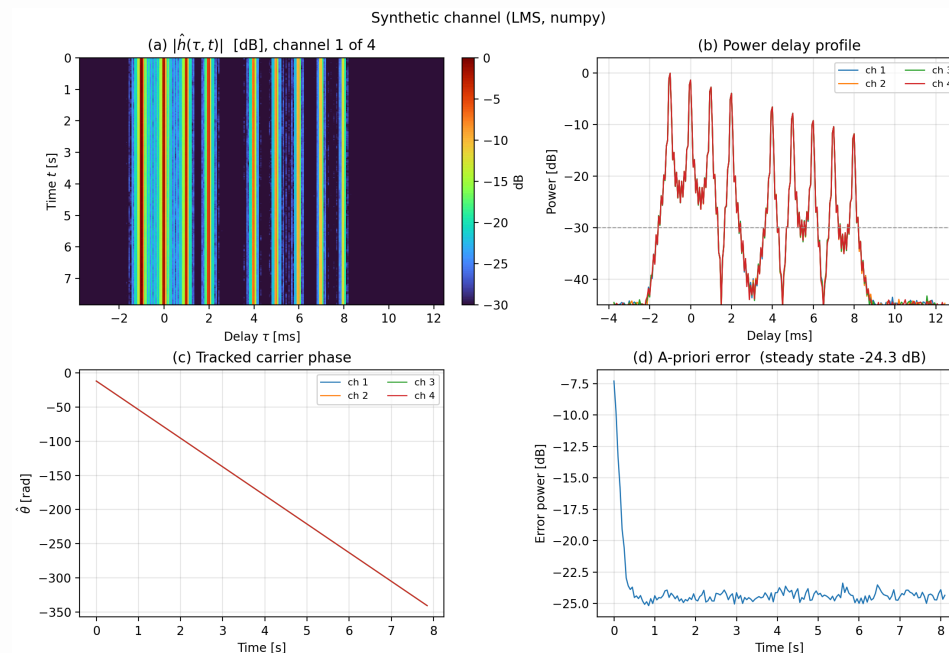
Before pointing the estimator at a recording whose answer you do not know, point it at one you do.

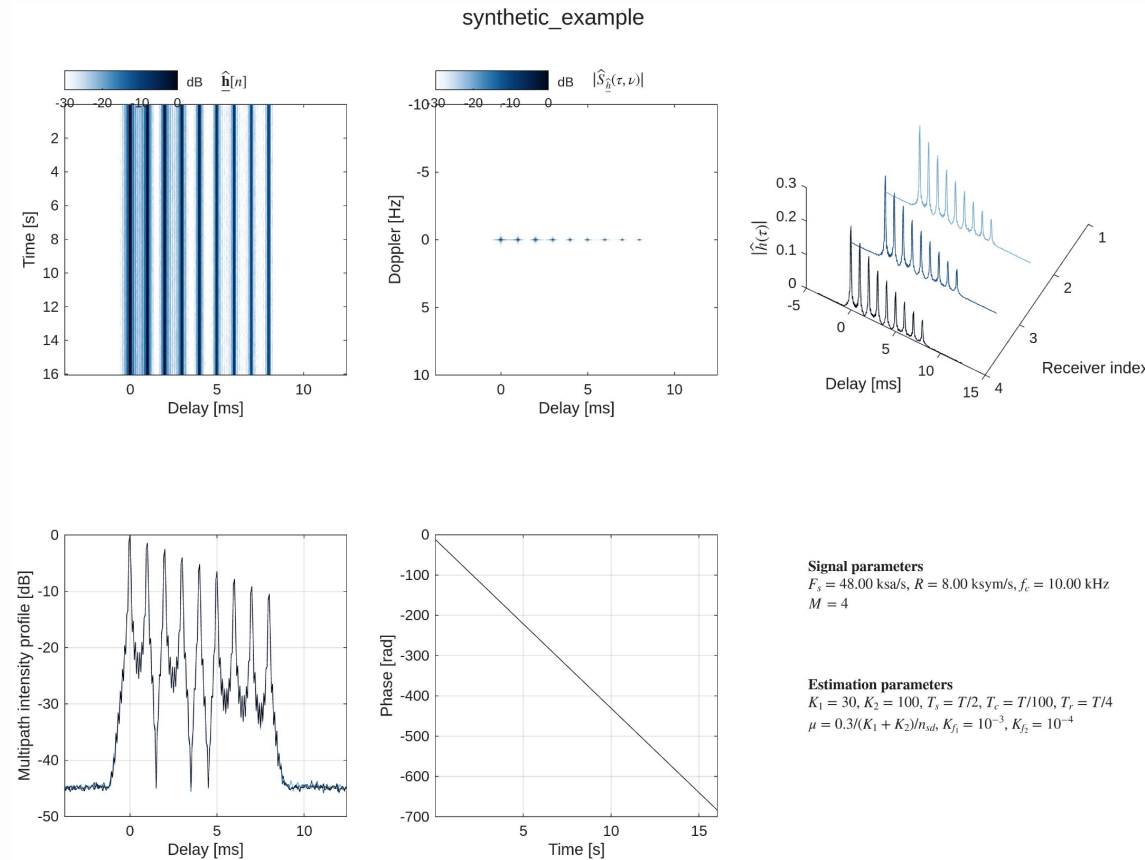
main.m builds a synthetic channel — nine paths over 10 ms, a geometric gain taper, random signs, a known time scaling $a = v/c$ — runs est, saves in library format, and plots.

Run from this room, just now:

True v/c : -667 ppm
Estimated v/c : -667 ppm
Multipath spread: 9.75 ms
Residual error: -24.3 dB

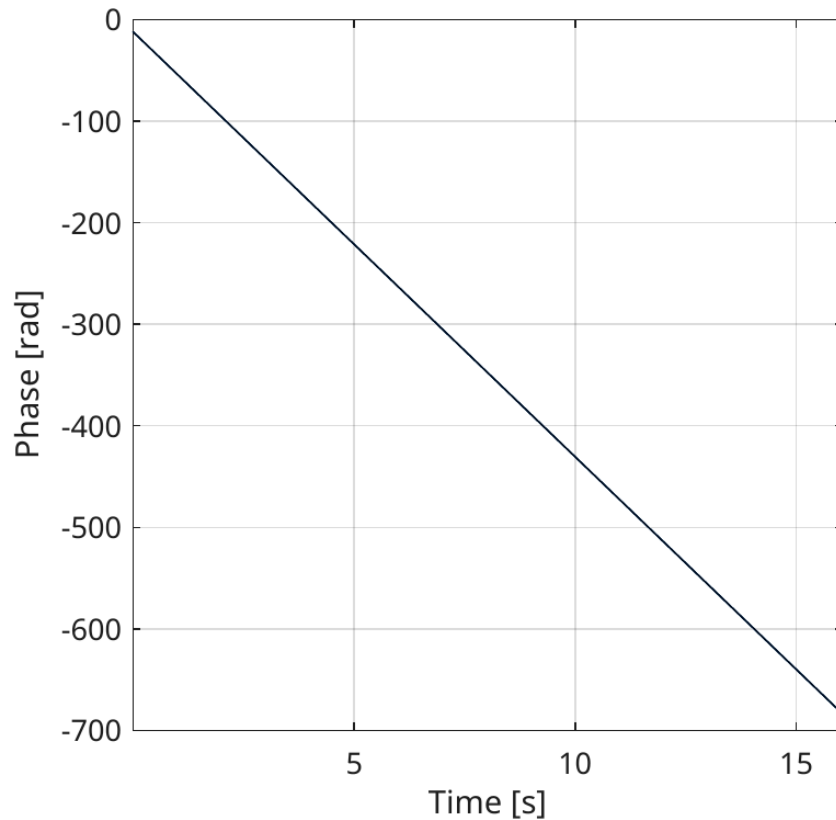
Nine paths over 10 ms, recovered to 9.75 ms of spread above -30 dB, and the Doppler to the last digit printed.





`plots(h_hat, theta_hat, params, meta, true)` — impulse response, scattering function, per-element response, multipath intensity profile, tracked phase, and the parameters the estimate was made with.

The panel to look at first



The PLL phase. A straight ramp means a constant radial velocity and a locked loop. Its slope is the Doppler:

$$\frac{v}{c} = -\frac{1}{2\pi f_c} \frac{d\hat{\theta}}{dt}$$

- **A clean ramp** — the loop is tracking. Read the speed off it.
- **A curve** — the platform is accelerating. Still fine.
- **A staircase, or noise** — the loop is not locked. Nothing downstream is meaningful.

In the synthetic run above the ramp reaches -700 rad in 16 s, which is the -667 ppm that was put in.

Is it a channel, or is it noise?

A matched filter will synchronize to **whatever sample happened to be tallest**, and an adaptive filter will fit noise happily and produce a plausible-looking picture. So there has to be a test, and the a-priori error is it.

Locked

$\text{residual} \leq -15 \text{ dB}$

The filter is explaining the recording. The synthetic check reaches -24 dB ; a good room recording reaches -17 to -19 .

Not locked

$\text{residual} > -8 \text{ dB}$

The filter is fitting noise. The picture will still look like a channel. **Do not publish it.**

Check it before you look at the pretty plot, not after

This was an observed failure, not a hypothetical one: 120 s of room tone with no probe in it at all still produced a synchronization peak and a delay-Doppler plot.

The live demonstration



A phone, a laptop, and the same pipeline

The estimator does not know it is underwater. A BPSK probe leaves a phone's loudspeaker, bounces around this room, and arrives at the laptop microphone — and the same `est` returns $\hat{h}(\tau, t)$ for the room.

Setting	Value
Waveform	BPSK, 4 kBaud on a 12 kHz carrier, 48 kHz sampling — occupying 10–14 kHz
Why that band	It is the flattest part of a laptop speaker-to-microphone path. Measured, not guessed.
Sequence	Fixed seed 0x0CEA45, so the receiver regenerates it and the WAV never has to come back
Estimator	Exactly the settings above, $M = 1$

Python port of `est.m` — NumPy and JAX backends agree to about 10^{-13} .

What it measures

Run	Residual	Clock drift
Synthetic channel, known truth	−21.7 dB	exact to 6 d.p.
Laptop speaker → laptop microphone (one clock)	−18.8 dB	0.0 ppm
Phone → laptop, stationary	−17.5 dB	+11.0 ppm
Phone → laptop, moving by hand	−1.5 dB	no lock

That +11 ppm is not a bug

It is the difference between two crystal oscillators that have never met. The delay tracker follows it exactly the way it follows a ship's Doppler at sea — the mechanism does not care which one it is looking at.

And when it fails, it fails for a reason

Pick the phone up and the residual climbs from -17.5 dB to -1.5 dB within half a second, and never recovers. Widening the tracking loops does not rescue it.

Why. At 12 kHz the wavelength is 29 mm, so **14 mm of movement inverts a path's phase**. A reverberant room delivers twenty-odd arrivals from twenty directions, each taking a different Doppler shift as the phone moves.

The delay tracker removes a **common** time scaling. It cannot remove twenty different ones.

And that is exactly the difference between a room and the ocean. At sea, a few well-separated arrivals share a Doppler dominated by platform motion, which is what makes joint delay-and-phase tracking work on **blue** and not here.

For the demo

Centimeters per second, or close range so the direct path dominates.

Your turn



Start with the synthetic check, then your own data

HANDS-ON · 10 minutes

1. **Everyone.** Clone `uwa-channels/estimate` and run `main.m`. Confirm the estimated v/c matches the truth and the residual is below -20 dB.
2. **Everyone.** Change `param.optim` from 1 to 3. Compare the transient and the time.
3. **Everyone.** Halve κ_2 . Watch the late arrivals leave the window and the residual rise.
4. **Brought a recording?** Fill in `data.Fs`, `R`, `fc`, `d`, `r`; set κ_2 from your delay spread; read the residual **before** the plot.
5. **Brought nothing?** Record the room and run the phone demo — a neighbor's phone will do.

We are back at 12:12

The question to answer: **did the filter lock, and how do you know?**

When it does not work

Symptom	Cause
No synchronization peak found	Wrong f_c , wrong R , or the probe is not in the recording.
Residual stuck above -8 dB	Not locked. Check f_c first, then whether d is really what you sent.
Arrivals walk off the edge of the window	κ_2 too small, or <code>delay_tracking</code> off on a mobile platform.
Phase is a staircase	κf_1 too small for the Doppler. Raise it, but expect noise.
Estimate is blurred in delay	μ too large, or λ too small. The filter is chasing noise.
It syncs to the wrong peak	With one transmission, take the strongest peak, not the first above threshold.
Python cannot read your saved file	Saved without <code>-v7.3</code> . Re-save, or open it with <code>load_channel</code> .

Contributing



From your estimate to the library

1. **Estimate** with `est`, and keep the `meta` the script writes. It is the record of how the number was made.
2. **Save** with `-v7.3 -nocompression`, with `h_hat`, one of `phi_hat`/`theta_hat`, `params`, `meta` and `version`.
3. **Check it round-trips**. Load it back, replay a probe through it, unpack it and look. If those three work, the format is right.
4. **Estimate the noise too**, if you have a quiet stretch of the recording. A channel without its noise model is half a contribution.
5. **Open a discussion** on the GitHub organization before you open a pull request — the channel files live on Zenodo, not in git, so contributing one is a conversation first.

What the maintainers will ask

Where and when, the geometry, the probe, and whether you can release it. Everything else is in `meta`.

1. `est` is **data-aided**: it needs the symbols you sent. Keep the seed.
2. It separates a slow impulse response from a fast phase, and optionally tracks the delay — which is what makes `phi_hat` channels compress.
3. κ_2 must cover your delay spread; μ scales with the filter length; LMS first, SFTF when the channel moves.
4. **The residual is the test.** Below -15 dB it is a channel. Above -8 dB it is noise wearing a channel's clothes.
5. Save with `-v7.3`, round-trip through `replay` and `unpack`, and you have a library file.

Section III of [arXiv:2609.03207](#) derives all of this — the basic estimator, the phase tracking, the delay tracking and the compression.

Wrap-up

Ten minutes: what to do next, and your questions.

Wrap-up

A Hands-On Tutorial on the Underwater Acoustic Channel Library

Zhengnan Li · Milica Stojanovic

Monday, September 21, 2026

OCEANS 2026 · Monterey Conference Center, Monterey, CA

12:20 – 12:30 · contribution workflow, discussions, Q&A

What you can do now that you could not at 08:30

1. **Choose a channel on purpose** — by the physical effect you want your receiver to confront, and by the limits your signal has to live inside.
2. **Drive any waveform you can write** through a measured ocean, on a real array, at an SNR you chose deliberately.
3. **Generate noise that is colored, correlated and — where the ocean is — impulsive**, instead of `randn`.
4. **Look at the channel itself**, and read motion off the phase.
5. **Turn your own at-sea recording into a file the same three functions accept.**

The point of all five modules

A result on a measured channel from this library is one somebody else can reproduce exactly — same file, same DOI, same code. That is the thing the field has been missing.

The five lines worth remembering

MATLAB / Octave

```
channel = load('blue_1.mat');  
noise   = load('blue_noise.mat');  
  
y = replay(x, fs, array_index, channel, start);  
w = noisegen(size(y), fs, array_index, noise);  
h = unpack(fs_time, array_index, channel);
```

Python

```
channel = load_channel("blue_1.mat")  
noise   = load_noise("blue_noise.mat")  
  
y = replay(x, fs, array_index, channel, start=n)  
w = noisegen(y.shape, fs, array_index, noise)  
h = unpack(fs_time, array_index, channel)
```

And the four habits that go with them:

- Take f_c and the element count **from the file**.
- Keep your signal inside the channel's band and shorter than the record.
- Pass `start` when the number matters.
- Set the noise level from a target SNR, not from a constant.

Contributing a channel

The path

1. Estimate with `est`, keeping `meta`.
2. Save `-v7.3 -nocompression`.
3. Round-trip it: `replay`, then `unpack`.
4. Fit a noise model from a quiet stretch.
5. Open a **Discussion**, not a pull request — the data lives on Zenodo, so it starts as a conversation.

What you will be asked

- Where, when, and by whom.
- The geometry: depths, range, array, spacing.
- The probe: sequence, modulation, f_c , R .
- Whether you can release it, and who to acknowledge.

Everything else is already in `meta`, which is exactly why `meta` exists.

Code contributions are easier

`matlab` and `python` take ordinary pull requests: branch from `main`, add a test, run CI. `CONTRIBUTING.md` in each repository has the workflow and the commit convention.

Where to ask, and where to look

Place	For
<code>uwa-channels.github.io</code>	What each channel is, and what is inside a file. Every page has Edit this page .
<code>github.com/uwa-channels/discussions</code>	Questions, design ideas, “is this channel right for my problem?”
Issues on matlab / python / estimate	Bugs, with a minimal reproducible example and your versions.
<code>doi:10.5281/ zenodo.21287414</code>	The data. Cite the version DOI in a paper; the concept DOI for the library.
<code>arXiv:2609.03207</code>	The paper: estimation, compression, noise models, and the verification against at-sea receptions.

Li, Chitre, Cuji, Preisig, Singer, Stojanovic, van Walree,
Underwater Acoustic Channel Library, arXiv:2609.03207, 2026.

Today's materials

Everything from this morning is archived at `uwa-channels.github.io/oceans26-tutorial`:

- These slides, all five modules.
- The template scripts and the reference solutions for both hands-on blocks.
- The figure-generation scripts — every plot you saw was produced by running the released toolbox on the released files, and you can re-run them.

Two things that would help us

Tell us what broke. A tutorial is the only time a hundred people install the same software on a hundred different laptops.

Tell us what is missing. Eight channels is not enough, and the gaps we know about — ice, very shallow rivers, high-frequency short range — are the ones somebody in this room has on a hard drive.

Questions

Zhengnan Li · Milica Stojanovic

`uwa-channels@ofdm.link`

With thanks to JAMSTEC, SFI Smart Ocean, the ATOC/AET team, and everyone who put a hydrophone in the water and kept the symbol sequence.